UART+FIFO+SENSOR

+stopwatch_watch

최기훈, 최민영, 조성재

2026.05.01~2026.05.06

1. 개요 (Overview)

1.1 목적 및 목표

1.2 설계 범위

1.3 프로젝트 요약

1.4 설계 사양 요약 (Specification Summary)

1.5 AS-IS / TO-BE

2. 프로젝트 관리 (Project Management)

2.1 일정 계획 (Schedule)

2.2 역할 분담 (Roles & Responsibilities)

2.3 설계 환경 (Design Environment)

3. 아키텍처 설계 (Architecture)

3.1 시스템 구조

3.2 설계 이론 및 배경 (Theory & Background)

4. 상세 설계 (Detailed Design)

4.1 Top-Level 통합 구조

4.1.1 top_final 구조

4.1.2 데이터 흐름 및 모드 선택 구조

4.2 UART 통신부 설계

4.3 FIFO 및 ASCII 처리부 설계

4.4 시간 기능부 설계

4.5 센서 기능부 설계

4.6 제어부 설계

4.7 출력부 설계

4.8 입력 안정화 및 보조 회로

5. 시뮬레이션 및 검증 (Simulation & Verification)

5.1 시뮬레이션 시나리오

5.2 Testbench

5.3 Waveform 분석

6. 결과 분석 및 트러블슈팅 (Analysis & trouble shooting)

6.1 FPGA Implementation 결과

6.2 Trouble Shooting

7. 결론

1. 개요 (Overview)

1.1 목적 및 목표

본 프로젝트의 목적은 FPGA 기반 UART 통신 시스템에 FIFO 구조와 다양한 Sensor Module을 통합하여, 외부와 실시간으로 데이터를 송수신할 수 있는 통합 디지털 시스템을 설계 및 구현하는 것이다.

기존의 Watch / Stopwatch 시스템을 확장하여 UART 기반 외부 인터페이스를 추가하였으며, Sensor 데이터 처리 기능을 포함한 Loopback 구조를 구현함으로써 입력부터 출력까지 전체 데이터 흐름을 검증 가능한 구조로 설계하였다.

주요 목표는 다음과 같다.

- UART 기반 Serial 통신 시스템 설계
- FIFO 기반 안정적인 데이터 버퍼링 구조 구현
- SR04 초음파 센서를 활용한 거리 측정 기능 구현
- DHT11 센서를 활용한 온습도 데이터 수집 기능 구현
- Watch / Stopwatch 시스템과 Sensor 데이터 통합
- ASCII 기반 데이터 송수신 구조 설계
- UART Loopback 구조를 통한 전체 시스템 검증
- FND 출력 및 UART 출력 동시 지원
- Button Debounce 기반 안정적인 입력 처리 구현

1.2 시스템 개요

본 시스템은 UART 통신 구조를 중심으로 FIFO, Sensor, Watch / Stopwatch 기능을 통합한 FPGA 기반 임베디드 시스템이다.

사용자는 UART를 통해 명령 데이터를 입력할 수 있으며, 입력된 데이터는 FIFO를 통해 안정적으로 저장 및 전달된다. 이후 ASCII Decoder를 통해 제어 신호로 변환되며, 내부 Control Unit이 Watch, Stopwatch, Sensor 동작을 제어한다.

또한 내부 데이터는 ASCII Sender를 통해 문자열 형태로 변환된 후 FIFO_TX를 거쳐 UART TX로 출력되며, 동시에 FND Display에도 표시된다.

시스템은 다음과 같은 주요 블록으로 구성된다.

- UART RX / TX
- FIFO_RX / FIFO_TX
- ASCII Decoder / ASCII Sender
- SR04 Controller (초음파 센서)
- DHT11 (온습도 센서)
- Stopwatch+Watch Datapath
- FND Controller
- Button Debounce
- Control Logic

1.3 프로젝트 요약

본 프로젝트에서는 UART 기반 입출력 구조와 FIFO 기반 데이터 흐름 제어를 중심으로 시스템을 설계하였다.

UART RX를 통해 입력된 ASCII 데이터는 FIFO_RX에 저장된 후 Decoder를 통해 제어 신호로 변환되며, 이를 기반으로 Watch / Stopwatch 및 Sensor 기능이 동작한다.

또한 내부 데이터는 ASCII Sender를 통해 문자열 데이터로 변환되어 FIFO_TX에 저장되고, UART TX를 통해 외부로 출력된다. 이를 통해 입력과 출력이 하나의 경로로 연결되는 UART Loopback 구조를 구현하였다.

Sensor 데이터와 시간 데이터는 FND를 통해 실시간으로 출력되며, UART를 통해 PC 환경에서도 확인 가능하도록 설계하였다.

본 시스템의 핵심 특징은 다음과 같다.

- UART 기반 외부 인터페이스 구현
- FIFO 기반 데이터 손실 방지 구조
- Sensor + Time System 통합 설계
- Loopback 기반 End-to-End 검증 구조
- FSM 기반 제어 구조 적용
- 모듈 단위 설계를 통한 확장성 확보

1.4 설계 사양 요약 (Specification Summary)

본 시스템은 FPGA 기반의 복합 디지털 시스템으로, UART 통신, FIFO 버퍼, Stopwatch/Watch, SR04 초음파 센서, DHT11 온습도 센서, 그리고 FND 출력 기능을 하나의 구조로 통합하여 구현하였다. 주요 설계 사양은 다음과 같다.

- 동작 클럭: 100MHz System Clock
- UART 통신 속도: 9600 bps
- UART 샘플링 방식: 16x Oversampling
- UART 데이터 형식: 1 Start Bit, 8 Data Bit, 1 Stop Bit
- FIFO 구조:
    - RX FIFO: UART 수신 데이터 임시 저장용
    - TX FIFO: ASCII 변환 데이터 송신 버퍼용
- 시간 데이터 해상도:
    - msec: 0~99
    - sec: 0~59
    - min: 0~59
    - hour: 0~23
- Stopwatch 기능:
    - RUN / STOP / CLEAR 지원
    - Up / Down Count 모드 지원
    - Record / View 기능 지원
- Watch 기능:
    - 시 / 분 / 초 설정 지원
    - 자동 시간 증가 기능 지원
- SR04 센서 기능:

    - Trig / Echo 기반 거리 측정
    - Echo pulse width 기반 거리 계산
    - 거리 데이터 3-digit 형태 출력
- DHT11 센서 기능:
    - 단일 데이터 라인 기반 온습도 수집
    - 40bit 데이터 수신 및 checksum 검증
    - 온도 / 습도 데이터 4-digit 형태 출력
- 출력 인터페이스:
    - 4-digit 7-Segment FND
    - LED 상태 표시
    - UART ASCII 상태 출력
- 입력 인터페이스:
    - Push Button 입력(btnR, btnL, btnU, btnD)
    - Switch 입력(sw[3:0])
    - UART 명령 입력(R, L, U, D, M, S)
- 입력 안정화:
    - Button Debounce 적용
    - UART 명령을 내부 제어 신호로 변환하는 ASCII Decoder 적용
- 설계 방식:
    - FSM 기반 제어 구조
    - 모듈화된 입력 / 처리 / 출력 분리 설계
    - FND 출력과 UART 출력의 동시 지원 구조

1.5 AS-IS / TO-BE

- AS-IS

  기존에는 Stopwatch/Watch 기능, 센서 기능, 그리고 UART 출력 기능이 각각 독립적으로 동작하는 구조로 접근하였다. 이 경우 시간 데이터와 센서 데이터를 별도의 경로에서 관리해야 하므로, 출력 자원과 제어 구조가 분산되고 시스템 전체 흐름을 한 번에 통합하기 어려운 문제가 있었다. 또한 버튼 입력과 UART 입력이 개별적으로 처리될 경우 동일 기능에 대한 제어 경로가 이중화될 수 있어, 구조적 복잡도가 증가하는 한계가 있었다.
- TO-BE

  본 설계에서는 Stopwatch/Watch, SR04, DHT11, UART 통신, FIFO, FND 출력을

하나의 FPGA 기반 시스템으로 통합하였다. 입력 계층에서는 버튼 입력과 UART ASCII 명령을 공통 제어 신호로 변환하도록 구성하였고, 처리 계층에서는 시간 데이터와 센서 데이터를 각 기능 블록에서 독립적으로 생성하도록 설계하였다. 이후 출력 계층에서는 동일한 내부 데이터를 FND와 UART를 통해 동시에 활용할 수 있도록 구조를 통합하였다.

주요 개선 사항은 다음과 같다.

- UART Rx/Tx와 FIFO를 포함한 데이터 흐름 구조를 추가하여 외부 터미널 기반 제어 및 상태 확인이 가능하도록 확장함
- Stopwatch/Watch와 Sensor 기능을 하나의 상위 제어 구조 안에서 선택적으로 동작하도록 통합함
- FND 출력과 UART ASCII 출력이 동일한 내부 데이터를 공유하도록 구성하여 출력 일관성을 확보함
- 버튼 입력과 UART 입력을 공통 FSM 제어 경로로 연결하여 제어 구조를 단순화함
- FIFO 기반 버퍼 구조를 적용하여 송수신 타이밍 차이로 인한 데이터 손실 가능성을 줄임
- 센서 데이터와 시간 데이터를 각각 독립 모듈에서 생성하되, 상위 모듈에서 통합 관리하도록 설계하여 확장성과 유지보수성을 높임

즉, 기존의 개별 기능 중심 구조에서 벗어나, 본 설계에서는 입력-처리-출력이 유기적으로 연결된 통합형 디지털 시스템 구조로 개선하였다.

2. 프로젝트 관리 (Project Management)

2.1 일정 계획 (Schedule)

단계	기간	주요 작업
요구사항 정의	1일	UART 및 Sensor 기능 정의
아키텍처 설계	1일	Block Diagram 및 데이터 흐름 설계
UART / FIFO 설계	2일	UART RX/TX 및 FIFO 구현
Sensor 설계	2일	SR04 및 DHT11 Controller 구현
시스템 통합	2일	Stopwatch/Watch 및 UART 통합
검증 및 디버깅	2일	Simulation 및 오류 수정
FPGA 적용	1	Basys3 Implementation

2.2 역할 분담 (Roles & Responsibilities)

담당 업무	담당자
UART RX / TX 설계	최기훈
FIFO 설계	최기훈
ASCII Decoder / Sender 설계	최기훈
DHT11 Sensor 설계	최민영
Watch / Stopwatch 설계	최민영
FND Controller 설계	최민영
SR04 Sensor 설계	조성재
UART / Sensor ASM 및 FSM 설계	조성재
발표 자료 구조화 및 시각화	조성재
Testbench 및 Simulation	공동
시스템 통합 및 디버깅	공동

2.3 설계 환경 (Design Environment)

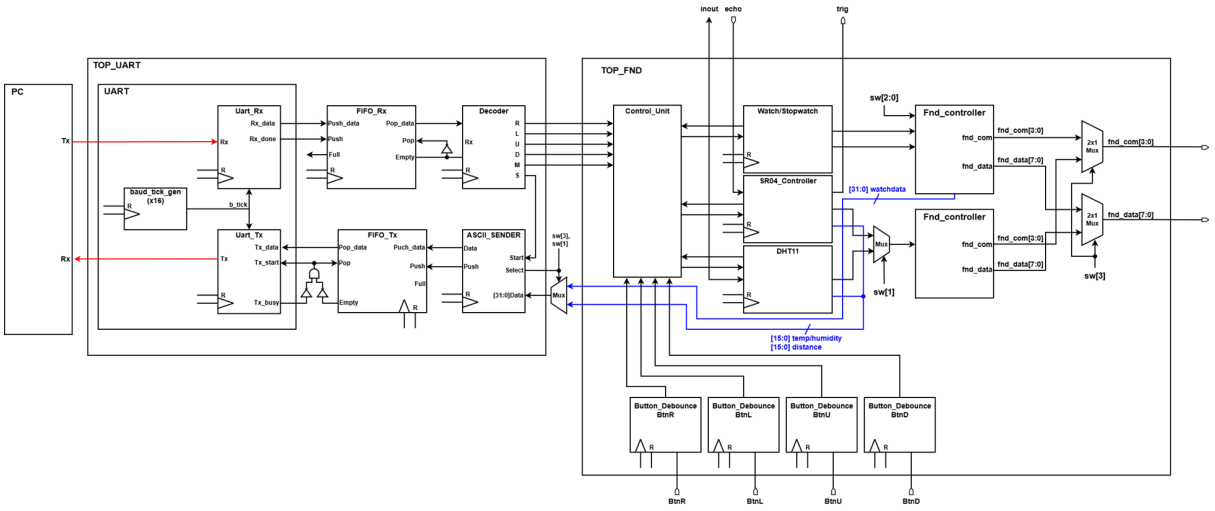
분류	내용
Language	Verilog HDL
FPGA	Basys3
EDA Tool	Vivado

Simulator	Vivado Simulator
Sensor	HC-SR04, DHT11
Interface Tool	ComPortMaster

3. 아키텍처 설계 (Architecture)

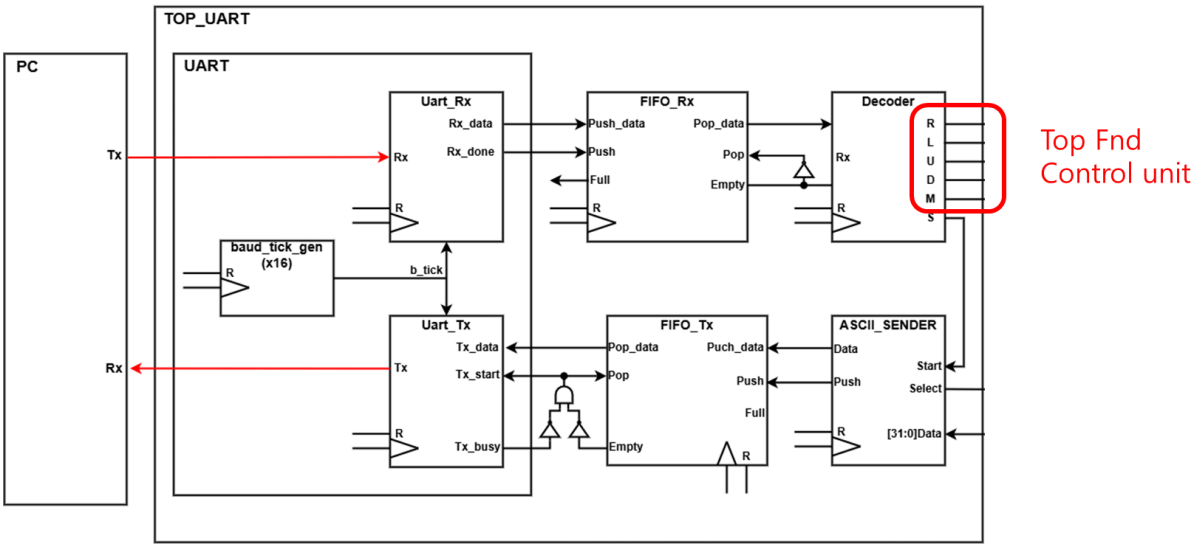
3.1 시스템 구조

- **System Block Diagram**

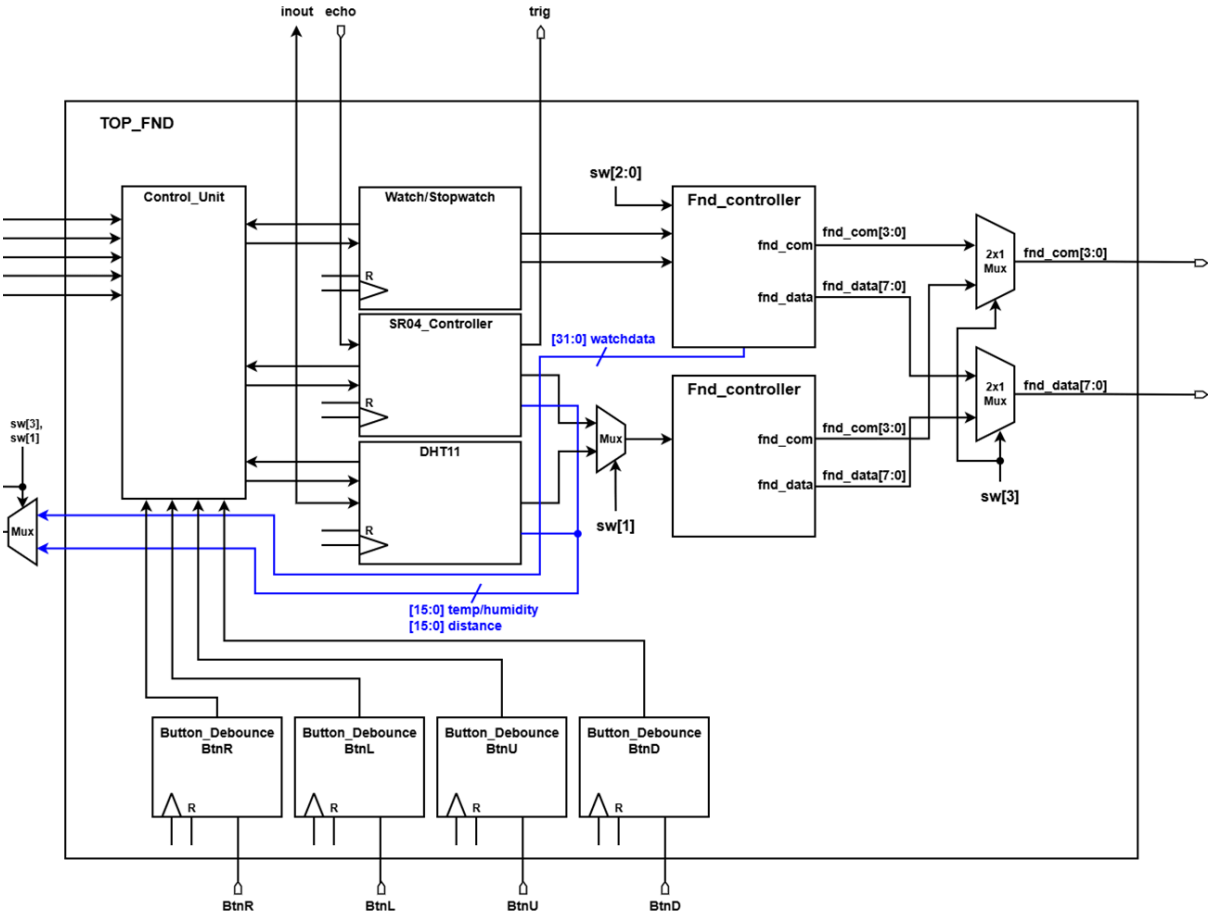


본 시스템은 UART 기반 데이터 흐름 구조를 중심으로 설계되었으며, 입력 → 처리 → 출력 흐름이 하나의 구조로 연결되도록 구현하였다.

- **Top Uart** (UART Loopback 구조)



- **Top Fnd**



전체 구조는 크게 Top UART Module과 Top FND Module의 두 계층으로 구성된다.

Top UART Module은 외부 PC 또는 시리얼 터미널로부터 입력되는 UART 데이터를 수신하고, 이를 내부 제어 신호 또는 상태 출력 데이터로 변환하는 역할을 수행한다. UART RX/TX는 직렬 데이터의 수신 및 송신을 담당하며, FIFO_RX와 FIFO_TX는 수신 데이터와 송신 데이터를 각각 버퍼링하여 송수신 타이밍 불일치로 인한 데이터 손실을 줄이도록 설계하였다. 또한 ASCII Decoder는 수신된 ASCII 명령을 버튼 제어 신호와 동일한 형태의 내부 제어 신호로 변환하고, ASCII Sender는 현재 시스템 상태 데이터를 ASCII 문자열로 변환하여 UART TX 경로로 전달한다. 이를 통해 사용자는 보드의 물리적 버튼뿐 아니라 UART 명령을 통해서도 동일한 기능을 제어할 수 있으며, 현재 상태 정보 역시 UART를 통해 외부로 확인할 수 있다.

Top FND Module은 실제 기능 동작과 표시를 담당하는 상위 블록으로 구성된다. 내부에는 Watch/Stopwatch 데이터 경로, SR04 초음파 센서 제어기, DHT11 온습도 센서 제어기, FND Controller, Sensor FND, Button Debounce, 그리고 전체 동작을 조정하는 Control Unit FSM이 포함된다. Watch/Stopwatch 블록은 시간 데이터 생성 및 제어를 담당하고, SR04와 DHT11 블록은 각각 거리 정보와 온습도 정보를 획득한다. 이후 이들 데이터는 상위 선택 신호에 따라 FND 출력 경로 또는 UART 출력 경로로 전달된다. 즉, 동일한 내부 데이터가 시각적 출력(FND)과 통신 출력(UART) 양쪽에서 공통적으로 활용될 수 있도록 구성하였다.

시스템의 데이터 흐름은 다음과 같이 정리할 수 있다. 먼저 외부 입력은 두 경로로 들어온다. 하나는 보드의 버튼 입력이며, 다른 하나는 UART를 통한 ASCII 명령 입력이다.

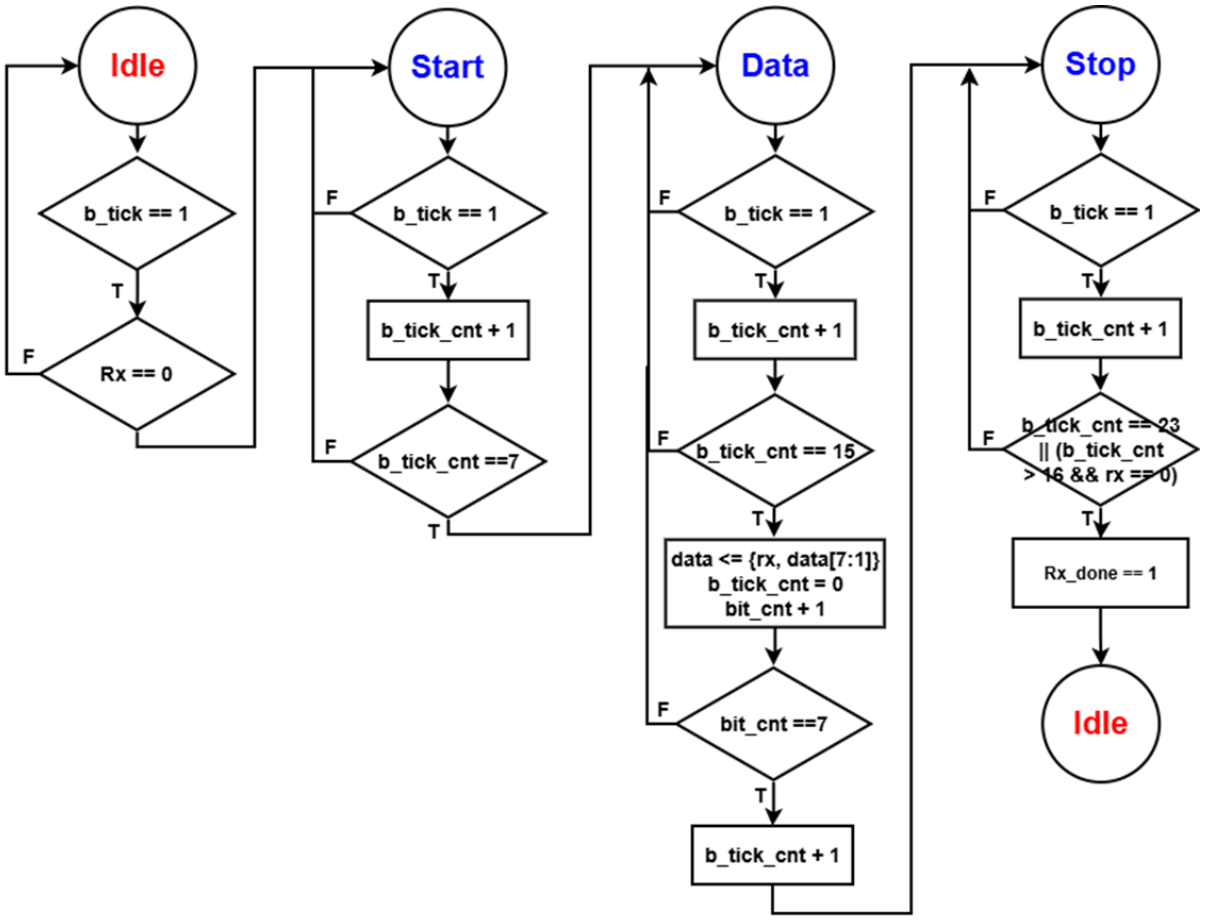
버튼 입력은 Button Debounce를 거쳐 안정화된 1-cycle pulse 형태로 변환 되고, UART 입력은 UART RX와 FIFO_RX를 거친 뒤 ASCII Decoder에서 내부 제어 신호로 변환된다. 이렇게 생성된 제어 신호는 Control Unit FSM으로 전달되어 현재 선택된 모드에 따라 Stopwatch/Watch, SR04, DHT11 중 해당 기능 블록을 동작시킨다. 이후 각 기능 블록에서 생성된 데이터는 FND Controller 또는 Sensor FND를 통해 표시되고, 동시에 필요 시 mux를 통해 UART 상태 출력용 데이터로도 전달된다. 최종적으로 UART TX 경로에서는 ASCII Sender와 FIFO_TX를 통해 상태 정보를 외부로 재전송한다.

본 구조의 핵심은 입력, 처리, 출력이 기능적으로 분리되어 있으면서도 상위 모듈에서 하나의 흐름으로 통합된다는 점이다. 입력 계층에서는 버튼과 UART 명령을 공통 제어 신호로 정규화하였고, 처리 계층에서는 시간 데이터와 센서 데이터를 각 기능 블록에서 독립적으로 생성하도록 하였으며, 출력 계층에서는 동일한 데이터를 FND와 UART로

동시에 활용할 수 있도록 구성하였다. 이와 같은 구조를 통해 모듈 간 역할이 명확해지고, 개별 기능 검증과 전체 시스템 통합 검증이 모두 용이한 형태의 계층적 아키텍처를 확보하였다.

3.2 설계 이론 및 배경

- **Uart_Rx ASM**



UART Rx FSM은 비동기 직렬 입력 데이터를 안정적으로 병렬 데이터로 복원하기 위해 IDLE → START → DATA → STOP의 4단계 상태 구조로 설계하였다.

IDLE 상태에서는 수신선(rx)이 High인 유휴 상태를 유지하다가, Start bit에 해당하는 Low 신호가 검출되면 START 상태로 전이한다. 이후 oversampling을 위해 생성된 16배 baud tick을 기준으로 타이밍을 계수하며, START 상태에서는 bit의 중앙 지점에 해당하는 시점(b_tick_cnt = 7)에서 Start bit를 확인한 뒤 DATA 상태로 진입하도록 하였다.

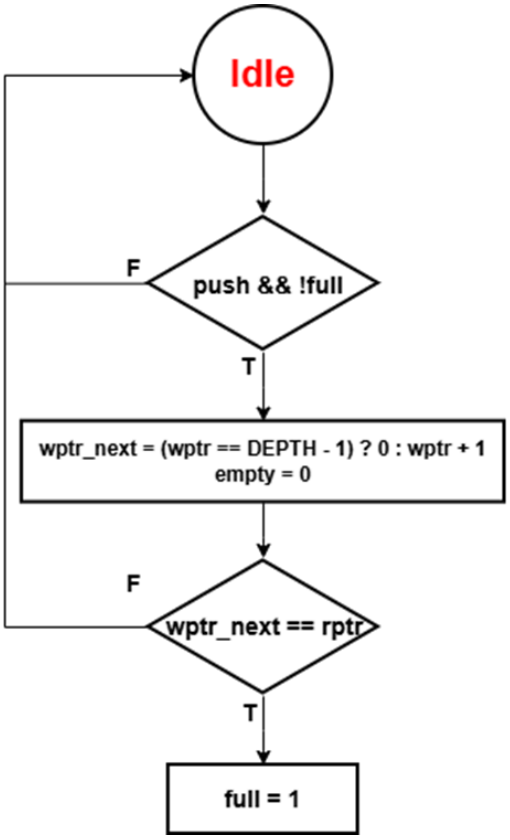
이러한 mid-sampling 방식은 bit 경계 부근의 전이 구간이 아닌 중앙부에서 데이터를 샘플링하게 하므로, 비동기 입력에서 발생할 수 있는 timing mismatch와 노이즈 영향을 줄이는 데 유리하다.

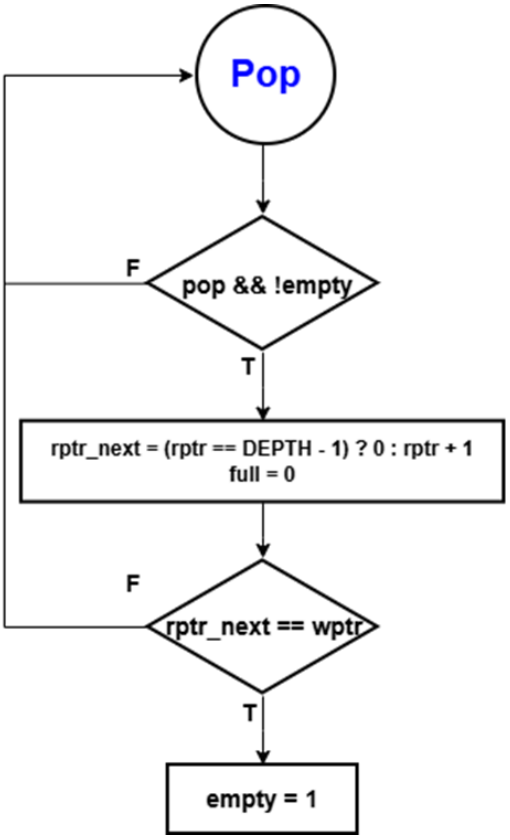
DATA 상태에서는 각 데이터 bit를 16개의 baud tick 주기로 샘플링하며, b_tick_cnt가 15에 도달할 때마다 현재 rx 값을 Shift Register에 저장한다. 이때 수신 데이터는 LSB first 방식으로 전송되므로, data_reg에 {rx, data_reg[7:1]} 형태로 순차 저장하여 최종적으로 8bit 병렬 데이터가 복원되도록 하였다.

또한 bit_cnt를 이용해 총 8개의 데이터 bit 수신 여부를 판단하며, 마지막 bit까지 저장이 완료되면 STOP 상태로 전이한다. STOP 상태에서는 stop bit 구간을 확인한 뒤 rx_done 신호를 발생시켜 상위 모듈이 해당 데이터를 유효 데이터로 인식할 수 있도록 하였다.

본 설계는 16x oversampling 구조를 적용함으로써 수신 타이밍 안정성과 데이터 복원 정확도를 향상시켰으며, UART 수신부의 신뢰성을 높이도록 구성하였다.

- **RX/TX_FIFO ASM**



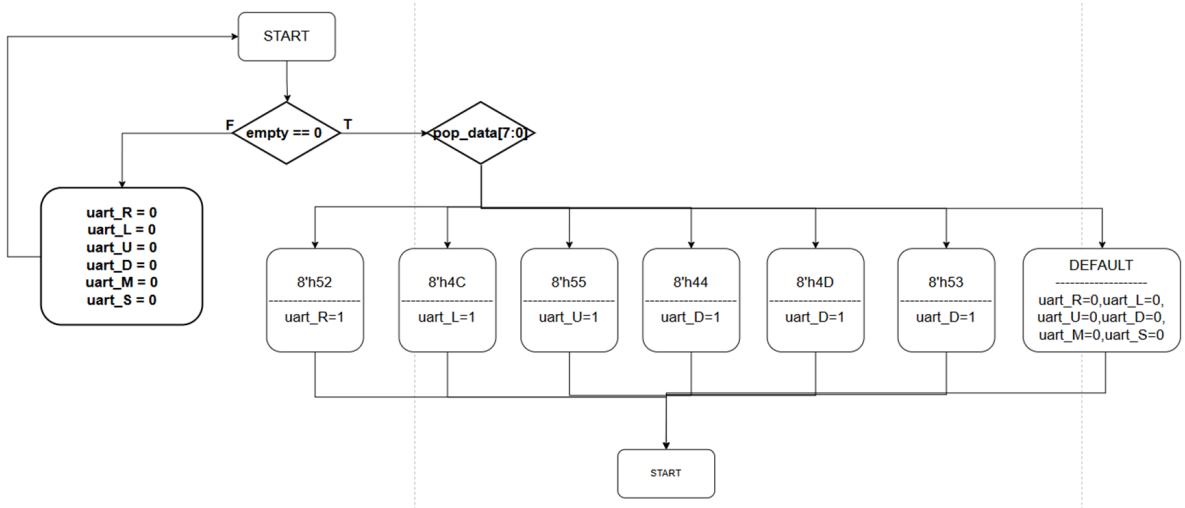


RX FIFO와 TX FIFO는 UART 송수신부와 상위 제어 로직 사이에서 데이터 버퍼 역할을 수행하도록 설계하였다. RX FIFO는 UART Rx에서 수신된 데이터를 임시 저장하여 상위 모듈이 안정적으로 읽을 수 있도록 하고, TX FIFO는 상위 모듈에서 생성된 송신 데이터를 임시 저장한 뒤 UART Tx가 전송 가능한 시점에 순차적으로 출력하도록 구성하였다.

FIFO는 push와 pop 조건에 따라 동작하며, push && !full 조건에서는 Write Pointer를 증가시키고, pop && !empty 조건에서는 Read Pointer를 증가시키는 방식으로 구현하였다. 이때 full 및 empty 상태를 함께 관리하여 버퍼가 가득 찬 상태에서의 추가 저장이나, 데이터가 없는 상태에서의 잘못된 읽기 동작을 방지하도록 하였다.

또한 Pointer는 Circular Buffer 구조를 따르도록 설계하여, DEPTH-1에 도달하면 다시 0으로 회귀하도록 구성하였다. 이를 통해 제한된 저장 공간을 반복적으로 활용할 수 있으며, Read Pointer와 Write Pointer를 독립적으로 관리함으로써 UART와 상위 로직 사이의 속도 차이가 존재하더라도 데이터 흐름이 안정적으로 유지되도록 하였다.

- **ASCII_Decoder ASM**

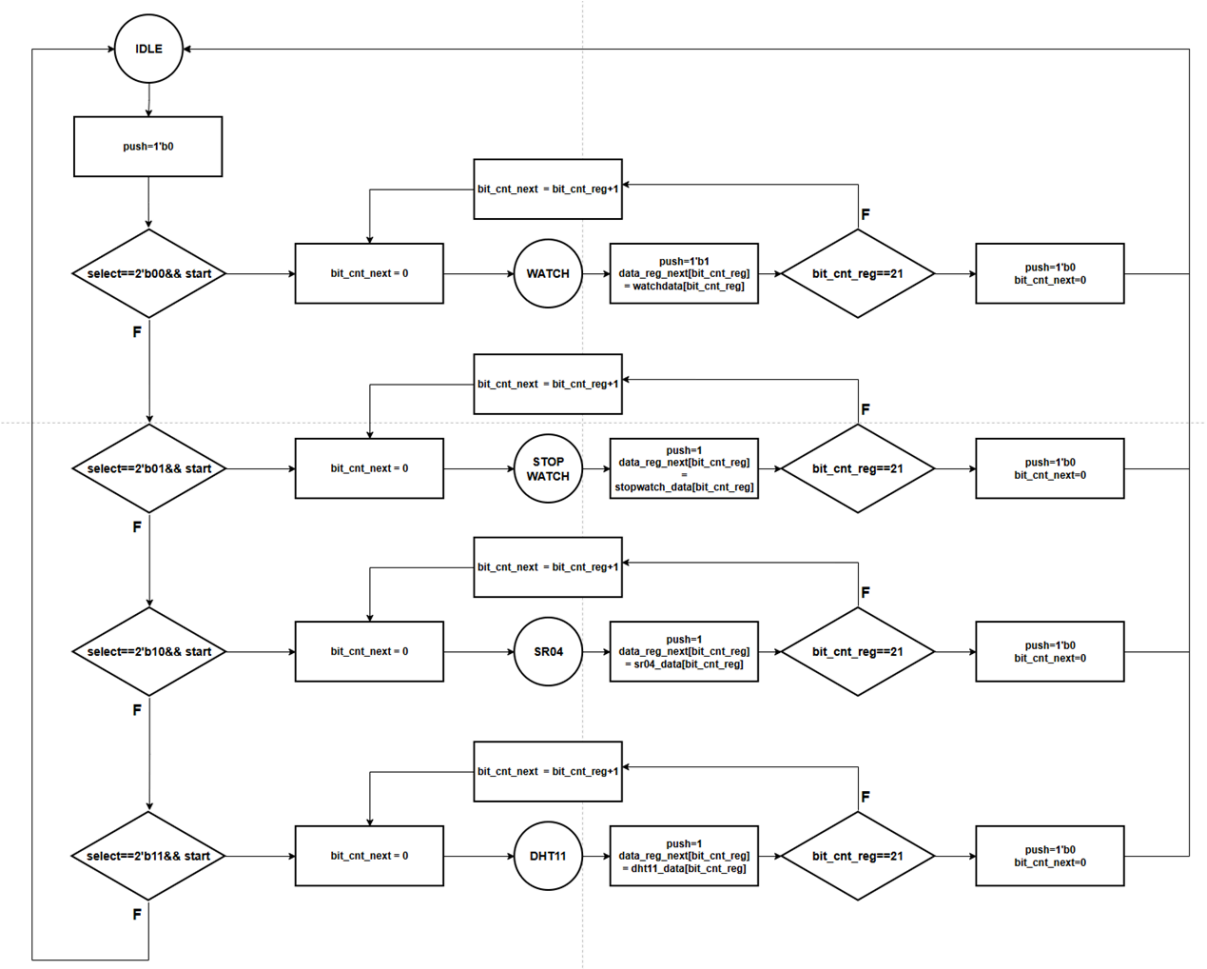


ASCII Decoder는 RX FIFO에서 Pop된 8-bit ASCII 데이터를 해석하여, 시스템 내부 제어 신호로 변환하는 역할을 수행한다. FIFO에 유효 데이터가 존재할 때(empty = 0)만 입력 데이터를 판별하며, 수신된 ASCII 값이 각각 8'h52, 8'h4C, 8'h55, 8'h44, 8'h4D, 8'h53인 경우 이에 대응하는 uart_R, uart_L, uart_U, uart_D, uart_M, uart_S 신호를 1로 출력하도록

설계하였다. 각 출력 신호는 1-cycle 명령 형태로 사용되며, 이를 통해 UART로 입력된 문자 명령을 실제 버튼 입력과 동일한 제어 신호로 변환할 수 있도록 하였다.

또한 정의되지 않은 ASCII 값이 입력되거나 FIFO가 비어 있는 경우에는 모든 출력 신호를 0으로 유지하도록 구성하여, 불필요한 오동작이 발생하지 않도록 하였다. 이를 통해 사용자는 UART 입력만으로도 시스템의 이동, 조작, 초기화 및 상태 요청 기능을 원격으로 수행할 수 있도록 구현하였다.

- **ASCII_Sender ASM**



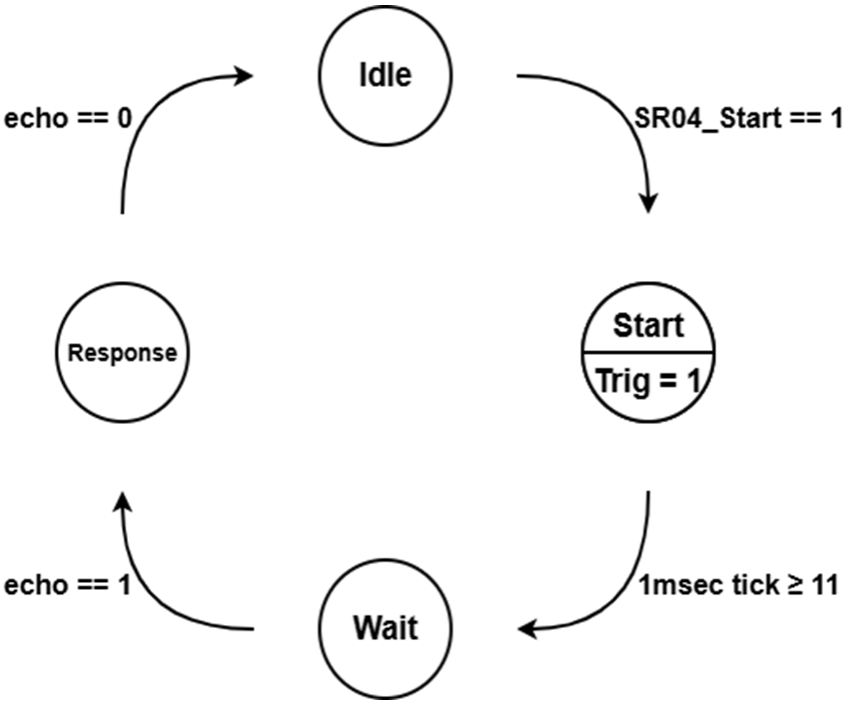
ASCII Sender는 현재 시스템 상태 데이터를 UART 송신용 ASCII 문자열로 변환하여 FIFO TX에 순차적으로 저장하는 역할을 수행한다. start 신호가 입력되면 select 값에 따라 WATCH, STOPWATCH, SR04, DHT11 중 하나의 출력 모드를 선택하며, 각 모드에 맞는 문자열 포맷을 구성한 뒤 한 문자씩 push_data로 생성하도록 설계하였다. 즉, 시간 데이터뿐

아니라 거리 데이터와 온습도 데이터도 동일한 송신 구조 안에서 ASCII 형태로 변환하여 외부 UART 터미널로 출력할 수 있도록 구현하였다.

상태 전이는 IDLE에서 시작하여, start 조건이 만족되면 선택된 모드 상태로 진입하고, 이후 bit counter를 증가시키면서 준비된 문자열 데이터를 한 바이트씩 FIFO TX에 저장하는 방식으로 동작한다. 예를 들어 WATCH 및 STOPWATCH 모드에서는 HH:MM:SS:MS 형식의 시간 정보를, SR04 모드에서는 거리값을, DHT11 모드에서는 온도 및 습도 정보를 각각 문자열 형태로 구성하여 전송한다. 모든 문자가 전송되면 push를 비활성화하고 bit counter를 초기화한 뒤 다시 IDLE 상태로 복귀하도록 설계하였다.

또한 문자열 전송 과정은 bit counter 기반의 순차 출력 구조로 구현하여, 정해진 포맷의 데이터가 항상 동일한 순서와 길이로 UART TX 경로에 전달되도록 하였다. 이를 통해 상위 모듈에서 생성된 내부 상태 데이터를 사람이 읽을 수 있는 ASCII 문자열로 안정적으로 변환하고, UART 기반 상태 모니터링 기능을 지원하도록 구성하였다.

- **SR04_Controller FSM**

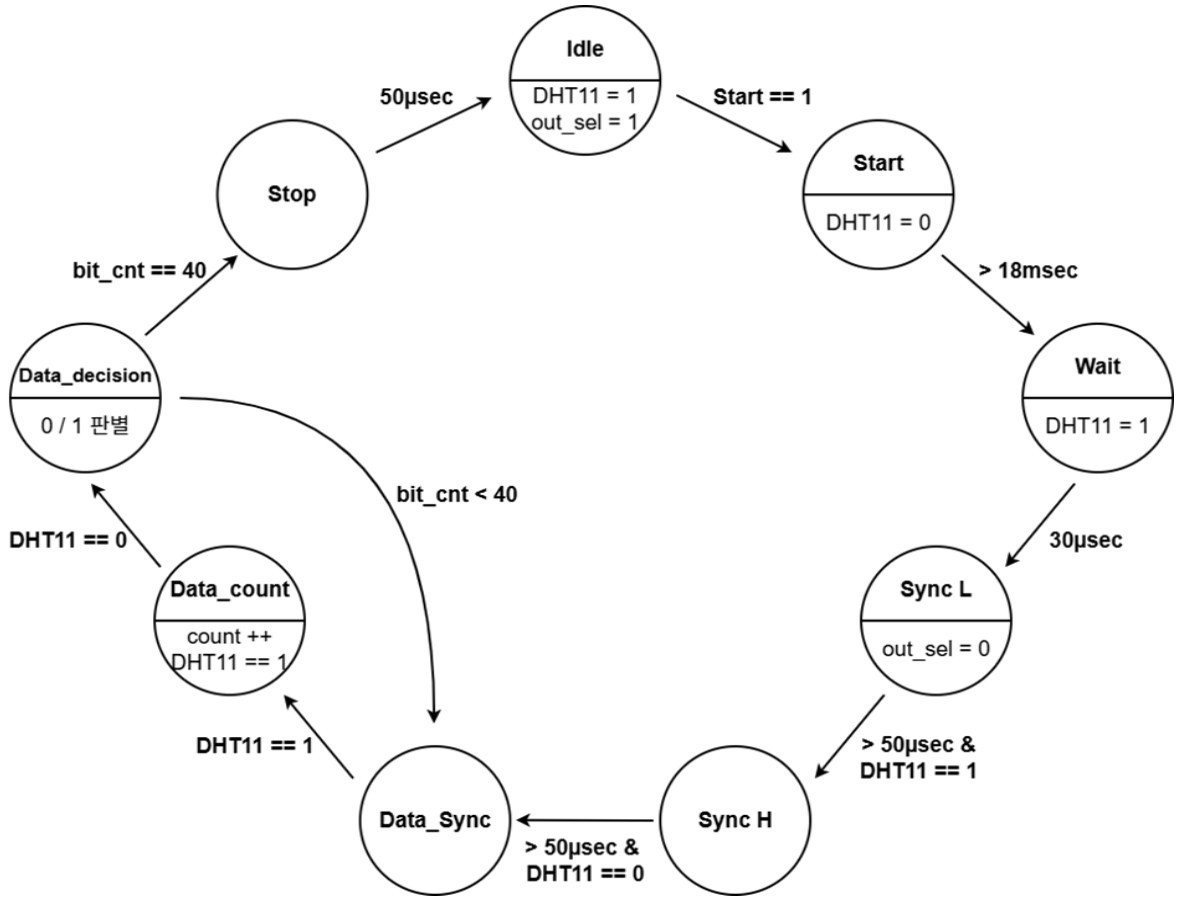


SR04 Controller FSM은 초음파 센서의 거리 측정을 위해 IDLE → START → WAIT → RESPONSE의 상태 구조로 설계하였다. IDLE 상태에서는 측정을 대기하다가 SR04_Start 신호가 입력되면 START 상태로 전이하며, 이 구간에서 Trig 신호를 일정 시간 동안 High로 출력하여 센서에 측정을 시작하도록 한다.

이후 WAIT 상태에서는 Echo 입력이 High가 되기를 대기하고, Echo가 검출되면 RESPONSE 상태로 전이한다. RESPONSE 상태에서는 Echo 신호가 High로 유지되는 시간을 counter로 측정하며, 이 시간을 기반으로 초음파의 왕복 시간을 계산하여 최종 거리 값으로 변환하도록 구현하였다.

즉, 본 FSM은 Trig pulse 생성, Echo 응답 대기, Echo pulse width 측정의 과정을 순차적으로 수행하는 구조이며, tick 기반 시간 측정 방식을 적용하여 FPGA 내부에서 실시간 거리 계산이 가능하도록 설계하였다.

- **DHT11 FSM**



DHT11 FSM은 단일 데이터 라인을 사용하는 온·습도 센서의 통신 프로토콜을 처리하기 위해 설계하였다. IDLE 상태에서는 대기 상태를 유지하다가 Start 신호가 입력되면 START 상태로 전이하며, 이 구간에서 FPGA가 데이터 라인을 Low로 일정 시간 유지하여 센서에 시작 조건을 전달한다. 이후 WAIT 상태에서는 라인을 다시 High로 복귀시킨 뒤 센서의 응답을 대기하도록 구성하였다.

센서 응답이 시작되면 SYNCL과 SYNCH 상태를 거쳐 초기 응답 구간을 검출하고, 이를 기준으로 실제 데이터 수신 타이밍을 동기화한다. 이 과정에서 출력 제어 신호를 비활성화하여 FPGA가 라인을 직접 구동하지 않고, 센서가 전달하는 응답 신호를 입력으로만 관찰하도록 설계하였다. 이를 통해 단일선 통신 구조에서 발생할 수 있는 출력 충돌을 방지할 수 있도록 하였다.

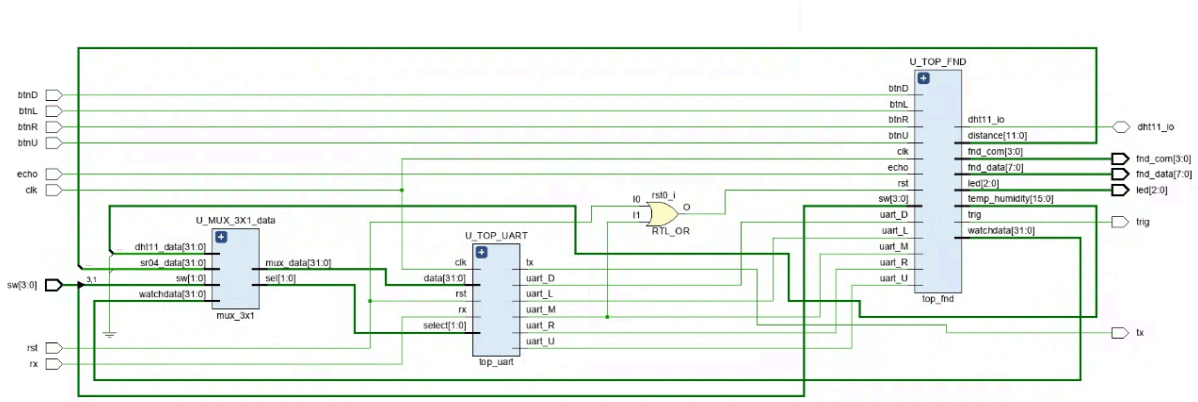
이후 DATA_SYNC 상태에서 각 비트의 시작 구간을 맞춘 뒤, DATA_COUNT 상태에서 입력 pulse의 High 유지 시간을 계수하고, DATA_DECISION 상태에서 해당 시간 길이를 기준으로 0 또는 1을 판별하도록 구현하였다. 이 과정을 반복하여 총 40bit 데이터를 순차적으로 수신하며, 수신 완료 후에는 humidity, temperature, checksum 데이터를 분리하여 저장한다.

마지막으로 STOP 상태를 거쳐 다시 IDLE 상태로 복귀하도록 구성하였으며, 수신된 데이터는 이후 온도 및 습도 값으로 활용되어 FND 출력과 UART 상태 출력에 공통으로 사용할 수 있도록 설계하였다. 이렇게 본 FSM은 시작 신호 생성, 센서 응답 동기화, pulse width 기반 데이터 판별, 데이터 저장까지의 전 과정을 순차적으로 수행하는 구조로 구현되었다.

4. 상세 설계 (Detailed Design)

4.1 Top-Level 통합 구조

- 4.1.1 top_final



top_final 모듈은 본 시스템의 최상위 통합 블록으로, UART 통신부와 FND 기반 기능부를 하나의 구조로 연결하는 역할을 수행한다. 외부로부터 clock, reset, UART 입력, 버튼 입력, 센서 입력, 스위치 입력을 받아 내부 기능 블록에 전달하고, 최종적으로 UART 송신, 센서 제어 신호, FND 출력, LED 출력을 생성하도록 설계하였다.

내부적으로는 크게 top_uart와 top_fnd의 두 상위 블록으로 구성된다. top_uart는 UART 명령을 해석하여 내부 제어 신호를 생성하고, 현재 시스템 상태 데이터를 ASCII 형태로 변환하여 UART TX로 출력한다. top_fnd는 Stopwatch/Watch, SR04, DHT11, FND 출력 기능을 포함하며, 실제 데이터 생성과 표시를 담당한다.

또한 top_final은 watchdata, distance, temp_humidity 신호를 이용하여 시간 데이터와 센서 데이터를 하나의 데이터 경로로 정리하고, 현재 선택된 모드에 따라 UART 상태 출력용 데이터로 전달한다. 이를 통해 동일한 내부 데이터를 FND와 UART에서 공통으로 활용할 수 있도록 구성하였다.

즉, top_final은 입력, 처리, 출력 블록을 하나로 통합하여 시스템 전체의 데이터 흐름과 기능 연결을 담당하는 최상위 모듈이다.

- 4.1.2 데이터 흐름 및 모드 선택 구조

본 시스템의 데이터 흐름은 입력, 처리, 출력의 3단계로 구분되도록 설계하였다. 입력 단계에서는 버튼 입력과 UART 명령 입력이 제어 신호로 변환되며, 센서 입력으로는 SR04의 echo 신호와 DHT11의 단일 데이터선 신호가 사용된다. 이 입력들은 control_unit_fsm과 각 센서 제어 블록으로 전달되어 현재 선택된 기능에 맞는 동작을 수행하도록 구성하였다.

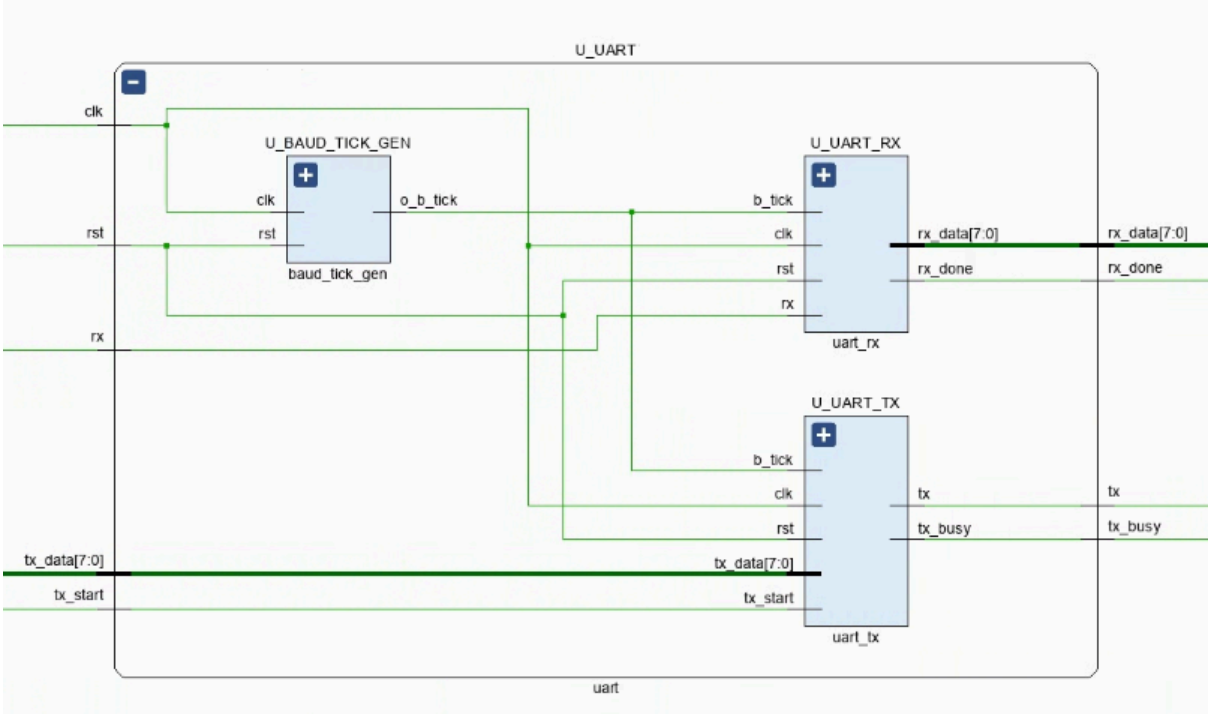
처리 단계에서는 Stopwatch/Watch 블록이 시간 데이터를 생성하고, SR04와 DHT11 블록이 각각 거리 데이터와 온습도 데이터를 생성한다. 생성된 데이터는 기능별로 독립적으로 유지되며, 상위 모듈에서 선택 신호에 따라 필요한 데이터만 출력 경로로 전달되도록 하였다. 이 구조를 통해 시간 기능과 센서 기능이 서로 간섭하지 않으면서 병렬적으로 존재할 수 있도록 설계하였다.

모드 선택은 스위치 조합을 기준으로 수행된다. sw[3]과 sw[1] 조합을 이용하여 Watch, Stopwatch, SR04, DHT11 중 현재 활성화할 기능을 결정하며, 이 선택 결과는 제어부와 데이터 경로에 동시에 반영된다. 즉, 어떤 모드가 선택되었는지에 따라 해당 기능 블록이 동작하고, 그에 대응하는 데이터만 FND 및 UART 출력 대상으로 사용된다.

출력 단계에서는 두 개의 경로가 사용된다. 첫 번째는 FND 출력 경로로, 시간 데이터는 main FND controller를 통해, 센서 데이터는 sensor FND를 통해 표시된다. 두 번째는 UART 출력 경로로, 선택된 내부 데이터가 ASCII Sender를 통해 문자열 형태로 변환된 뒤 UART TX로 전달된다. 따라서 본 시스템은 하나의 모드 선택 구조를 기준으로 동일한 내부 데이터를 FND와 UART 양쪽에서 동시에 활용할 수 있도록 구성하였다.

4.2 UART 통신부 설계

- 4.2.1 UART 전체 구조



UART 블록은 외부 터미널과 FPGA 시스템 사이의 명령 입력 및 상태 출력 기능을 담당하도록 설계하였다. 전체 구조는 top_uart를 중심으로 구성되며, 내부에는 UART 본체, RX FIFO, ASCII Decoder, ASCII Sender, TX FIFO가 포함된다. 이를 통해 UART 입력은 내부 제어 신호로 변환되고, 반대로 시스템 상태 데이터는 ASCII 문자열로 변환되어 외부로 송신되도록 구성하였다.

수신 경로는 rx 입력으로부터 시작된다. 외부에서 들어온 직렬 데이터는 UART Rx에서 8-bit 병렬 데이터와 rx_done 신호로 변환되며, 이후 RX FIFO에 저장된다. FIFO에 저장된 데이터는 ASCII Decoder로 전달되어, 문자 명령(R, L, U, D, M, S)에 따라 내부 제어 신호로 변환된다. 이를 통해 UART 입력만으로도 버튼 입력과 동일한 방식의 시스템 제어가 가능하도록 하였다.

송신 경로는 시스템 내부 상태 데이터를 외부로 전달하기 위한 구조이다. 상위 모듈에서 전달된 select와 data 신호를 기반으로 ASCII Sender가 현재 모드에 맞는 문자열을 생성하고, 이를 TX FIFO에 순차적으로 저장한다. 이후 TX FIFO에 데이터가 존재하고 UART Tx가 idle 상태일 때, 해당 데이터가 UART Tx로 전달되어 직렬 출력(tx)으로 송신되도록 설계하였다.

즉, 본 UART 구조는 단순 loopback이 아닌, 입력 측에서는 명령 해석 기능을 수행하고 출력 측에서는 상태 모니터링 기능을 수행하는 양방향 통신 구조이다. 또한 RX FIFO와 TX FIFO를 각각 분리하여 수신 데이터와 송신 데이터를 독립적으로 관리함으로써, UART와 상위 제어 로직 간의 속도 차이가 존재하더라도 데이터 흐름이 안정적으로 유지되도록 하였다.

- 4.2.2 UART Rx

UART Rx FSM은 비동기 직렬 입력 데이터를 안정적으로 병렬 데이터로 복원하기 위해 IDLE, START, DATA, STOP의 4개 상태로 설계하였다. IDLE 상태에서는 rx 라인이 High인 유휴 상태를 유지하다가, Start bit에 해당하는 Low 신호가 검출되면 START 상태로 전이한다. 이후 oversampling을 위해 생성된 16배 baud tick을 기준으로 timing을 계수하며, Start bit의 중앙 시점에서 수신 시작을 확인한 뒤 DATA 상태로 진입하도록 구성하였다.

DATA 상태에서는 각 bit를 16 tick 주기로 샘플링하며, 샘플링된 값은 shift register 구조를 이용해 순차적으로 저장된다. UART 데이터는 LSB first 방식으로 전송되므로, 수신된 rx 값을 기존 데이터와 함께 shift하여 8-bit 병렬 데이터로 복원하도록 설계하였다. 또한 bit counter를 이용해 총 8개의 data bit 수신 여부를 판별하고, 마지막 bit까지 저장이 완료되면 STOP 상태로 전이한다.

STOP 상태에서는 stop bit 구간을 확인한 뒤 rx_done 신호를 발생시켜 상위 모듈이 해당 데이터를 유효 데이터로 인식할 수 있도록 하였다. 이와 같은 구조를 통해 UART Rx FSM은 start bit 검출, 중앙 시점 샘플링, 8-bit 데이터 복원, 수신 완료 신호 생성의 과정을 순차적으로 수행하며, 16x oversampling 기반으로 수신 timing 안정성과 데이터 복원 정확도를 높이도록 설계하였다.

- 4.2.3 UART Tx

UART Tx FSM은 8-bit 병렬 데이터를 비동기 직렬 데이터로 변환하여 송신하기 위해 IDLE, START, DATA, STOP의 4개 상태로 설계하였다. IDLE 상태에서는 tx 라인을 High로 유지하며 대기하다가, tx_start 신호가 입력되면 송신할 데이터를 내부 register에 저장하고 START 상태로 전이한다. 이때 tx_busy 신호를 활성화하여 현재 송신이 진행 중임을 상위 모듈에 알리도록 구성하였다.

START 상태에서는 UART 프로토콜에 따라 start bit인 0을 출력하며, baud tick 기준으로 1bit 구간이 유지되도록 하였다. 이후 DATA 상태에서는 내부 register에 저장된 데이터를 LSB

first 방식으로 1bit씩 순차 출력하고, 매 bit 전송 이후 shift register를 이용해 다음 bit가 출력되도록 구성하였다. 또한 bit counter와 tick counter를 사용하여 총 8bit가 정확한 timing으로 전송되도록 제어하였다.

모든 데이터 bit 전송이 완료되면 STOP 상태로 전이하여 stop bit인 1을 출력하고, 1bit 구간이 끝난 뒤 다시 IDLE 상태로 복귀하도록 설계하였다. 이때 tx_busy 신호는 비활성화되어 새로운 데이터 전송이 가능함을 나타낸다. 이와 같은 구조를 통해 UART Tx FSM은 병렬 데이터 로드, start bit 출력, data bit 직렬화, stop bit 출력의 과정을 순차적으로 수행하며, baud tick 기반 timing 제어를 통해 안정적인 UART 송신이 가능하도록 설계하였다.

- 4.2.4 Baud Tick Generator

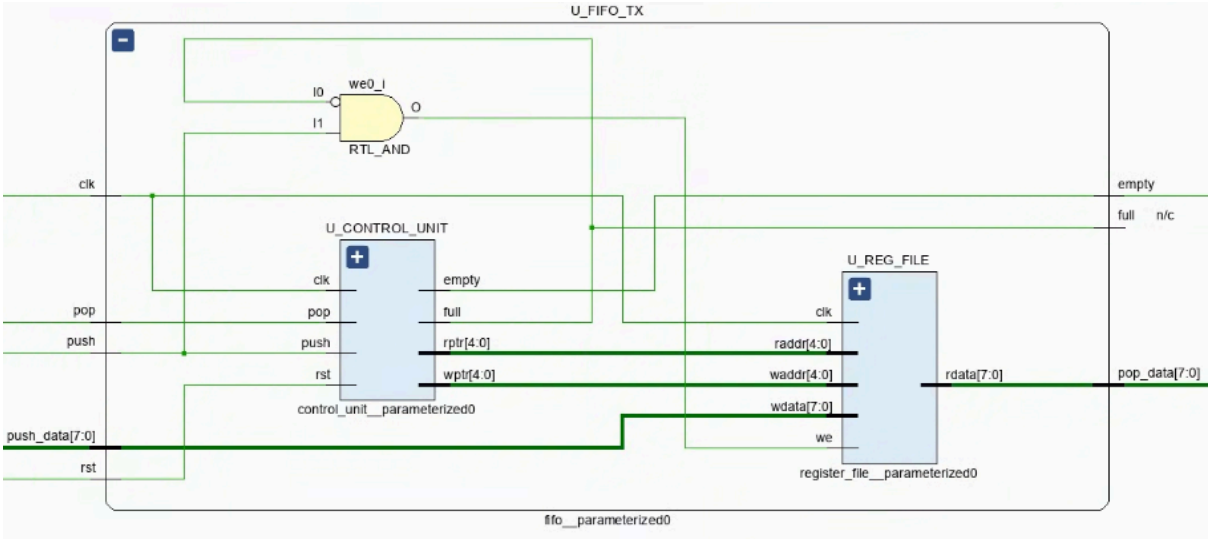
Baud Tick Generator는 UART 송수신부에서 공통으로 사용하는 기준 timing 신호를 생성하기 위해 설계하였다. 시스템 클럭은 100 MHz이므로, 이를 UART 통신 속도인 9600 bps에 맞는 16배 oversampling 주기로 분주하여 baud tick 신호를 생성하도록 구성하였다. 이 baud tick은 UART Rx와 UART Tx가 동일한 시간 기준으로 동작하도록 하는 핵심 timing 신호이다.

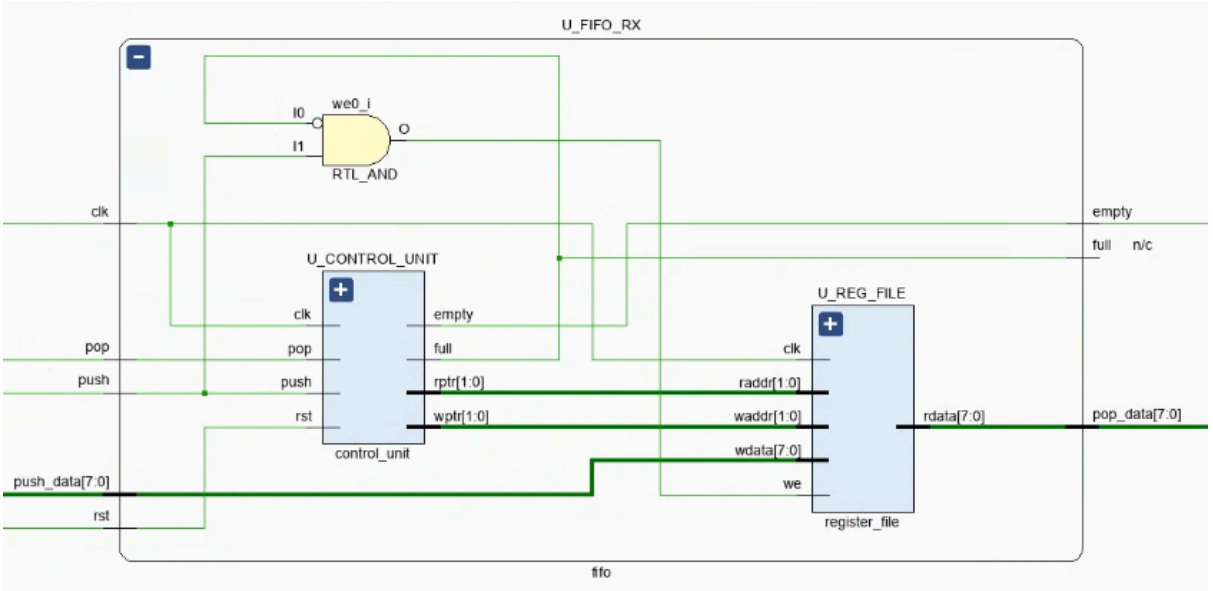
내부적으로는 counter를 이용한 분주 구조를 사용하였다. 클럭이 입력될 때마다 counter가 증가하고, 설정된 분주 값에 도달하면 counter를 0으로 초기화하면서 1-cycle 폭의 baud tick을 발생시킨다. 그렇지 않은 경우에는 baud tick을 0으로 유지하도록 설계하였다. 이를 통해 일정한 주기의 tick 신호가 반복적으로 생성되도록 하였다.

이렇게 생성된 baud tick은 UART Rx FSM에서는 bit 중앙 시점 샘플링과 data bit 수신 timing 제어에 사용되고, UART Tx FSM에서는 start bit, data bit, stop bit의 전송 구간을 제어하는 기준 신호로 사용된다. 즉, Baud Tick Generator는 UART 전체 구조에서 송신부와 수신부의 timing 기준을 통일하여 데이터 송수신의 안정성을 확보하는 역할을 수행한다.

4.3 FIFO 및 ASCII 처리부 설계

- 4.3.1 Rx/Tx FIFO 구조





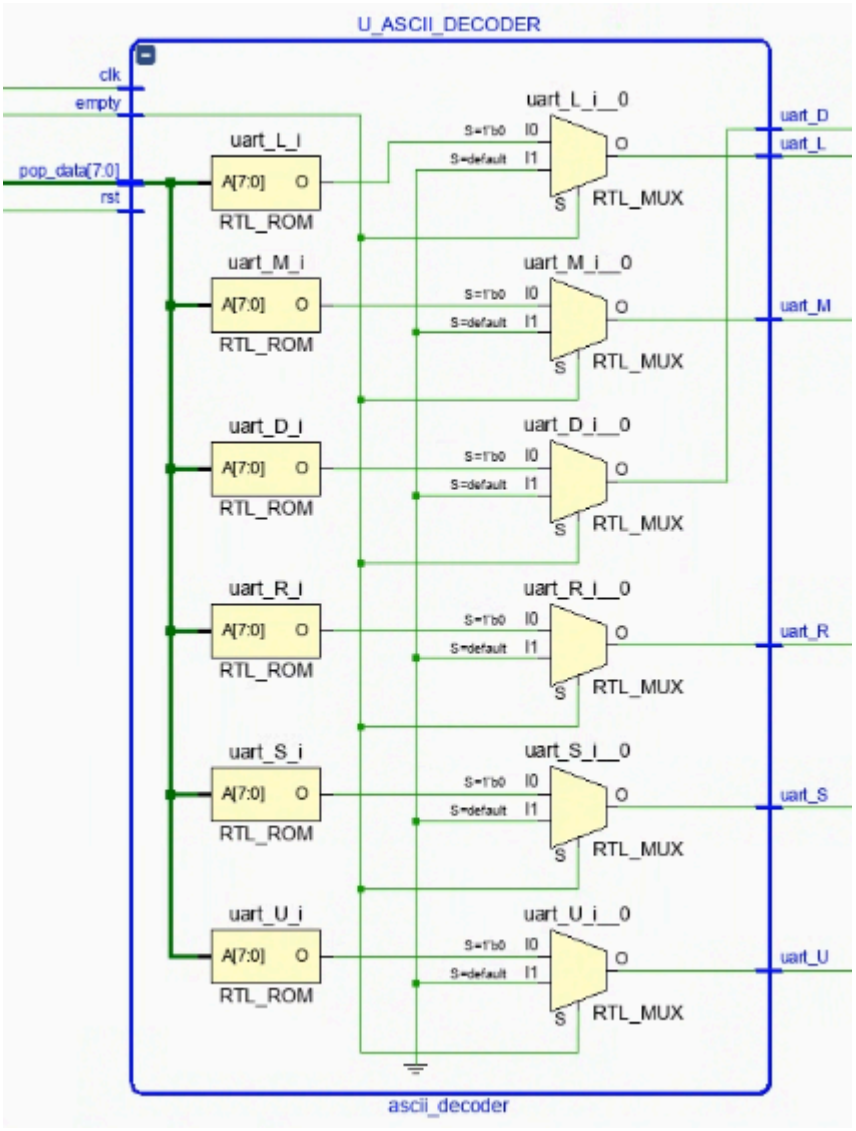
RX FIFO와 TX FIFO는 UART 송수신부와 상위 제어 로직 사이에서 데이터 버퍼 역할을 수행하도록 설계하였다. RX FIFO는 UART Rx에서 수신된 데이터를 임시 저장하여 상위 모듈이 안정적으로 읽을 수 있도록 하고, TX FIFO는 ASCII Sender에서 생성된 송신 데이터를 임시 저장한 뒤 UART Tx가 전송 가능한 시점에 순차적으로 출력하도록 구성하였다. 이를 통해 송수신 timing이 서로 완전히 일치하지 않더라도 데이터 흐름이 안정적으로 유지되도록 하였다.

FIFO는 push와 pop 조건에 따라 동작하며, push && !full 조건에서는 Write Pointer를

증가시키고, pop && !empty 조건에서는 Read Pointer를 증가시키는 구조로 구현하였다. 또한 full 및 empty 상태를 함께 관리하여 버퍼가 가득 찬 상태에서의 추가 저장이나, 데이터가 없는 상태에서의 잘못된 읽기 동작을 방지하도록 하였다. 이 구조는 UART와 상위 로직 사이의 속도 차이로 인해 발생할 수 있는 데이터 손실이나 overrun 가능성을 줄이는 데 목적이 있다.

내부 저장 구조는 register file 기반으로 구현하였으며, Read Pointer와 Write Pointer를 독립적으로 관리하는 Circular Buffer 방식을 적용하였다. Pointer는 DEPTH-1에 도달하면 다시 0으로 회귀하도록 설계하여 제한된 저장 공간을 반복적으로 사용할 수 있도록 하였다. 결과적으로 RX/TX FIFO는 비동기적 데이터 흐름을 완화하고, UART 입력 명령 처리와 상태 출력 과정이 보다 안정적으로 동작할 수 있도록 하는 중간 버퍼 구조로 동작한다.

- 4.3.2 ASCII Decoder



ASCII Decoder는 RX FIFO에서 pop된 8-bit ASCII 데이터를 해석하여 시스템 내부 제어 신호로 변환하는 역할을 수행한다. UART를 통해 입력된 문자 명령을 실제 버튼 입력과 동일한 의미의 제어 신호로 바꾸어 줌으로써, 외부 터미널만으로도 시스템을 원격 제어할 수 있도록 설계하였다.

동작 방식은 비교적 단순하다. FIFO에 유효 데이터가 존재할 때만 입력 데이터를 판별하며, 수신된 ASCII 값이 각각 R, L, U, D, M, S에 해당하면 이에 대응하는 uart_R, uart_L, uart_U, uart_D, uart_M, uart_S 신호를 활성화하도록 구성하였다. 이 신호들은 상위 control unit으로 전달되어 버튼 입력과 동일한 제어 경로에서 사용된다.

또한 정의되지 않은 ASCII 값이 입력되거나 FIFO가 비어 있는 경우에는 모든 출력 신호를 0으로 유지하도록 설계하였다. 이를 통해 불필요한 오동작을 방지하고, 유효한 명령만 시스템 제어에 반영되도록 하였다. 결과적으로 ASCII Decoder는 UART 기반 문자 입력을 내부 명령 신호로 변환하는 인터페이스 블록으로 동작한다.

- 4.3.3 ASCII Sender

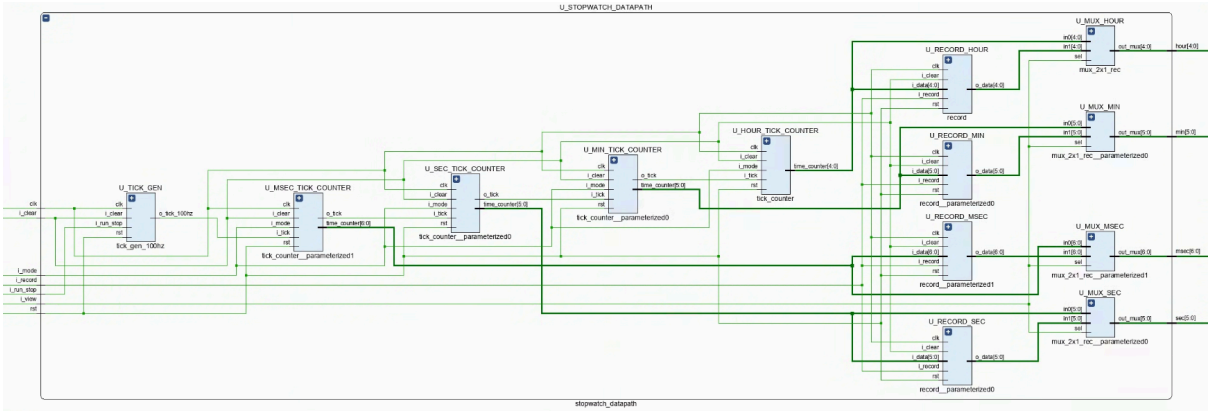
ASCII Sender는 현재 시스템 상태 데이터를 UART 송신용 ASCII 문자열로 변환하여 TX FIFO에 순차적으로 저장하는 역할을 수행한다. 상위 모듈에서 start 신호가 입력되면, 현재 선택된 모드에 따라 Watch, Stopwatch, SR04, DHT11 중 하나의 출력 형식을 결정하고, 해당 데이터를 사람이 읽을 수 있는 문자열 형태로 변환하도록 설계하였다.

문자열 생성은 FSM과 bit counter 기반 구조로 이루어진다. IDLE 상태에서 대기하다가 start 조건이 만족되면 선택된 모드 상태로 전이하고, 이후 counter 값을 증가시키면서 각 위치에 해당하는 ASCII 코드를 순차적으로 생성한다. 예를 들어 시간 데이터는 WATCH:HH:MM:SS:MS 또는 STOPWATCH:HH:MM:SS:MS 형식으로 구성되며, 센서 데이터는 거리값 또는 온도/습도 값을 포함하는 문자열 형태로 변환된다. 생성된 각 문자는 push_data로 출력되고, push 신호와 함께 TX FIFO에 저장되도록 하였다.

모든 문자가 전송되면 push를 비활성화하고 counter를 초기화한 뒤 다시 IDLE 상태로 복귀하도록 설계하였다. 이와 같은 구조를 통해 내부의 32-bit 또는 16-bit 데이터 값을 직접 전송하는 대신, UART 터미널에서 바로 확인 가능한 ASCII 문자열 형태로 출력할 수 있도록 하였다. 결과적으로 ASCII Sender는 시스템 내부 상태를 외부 사용자에게 직관적으로 전달하기 위한 상태 출력 인터페이스 블록으로 동작한다.

4.4 시간 기능부 설계

- 4.4.1 Stopwatch Datapath



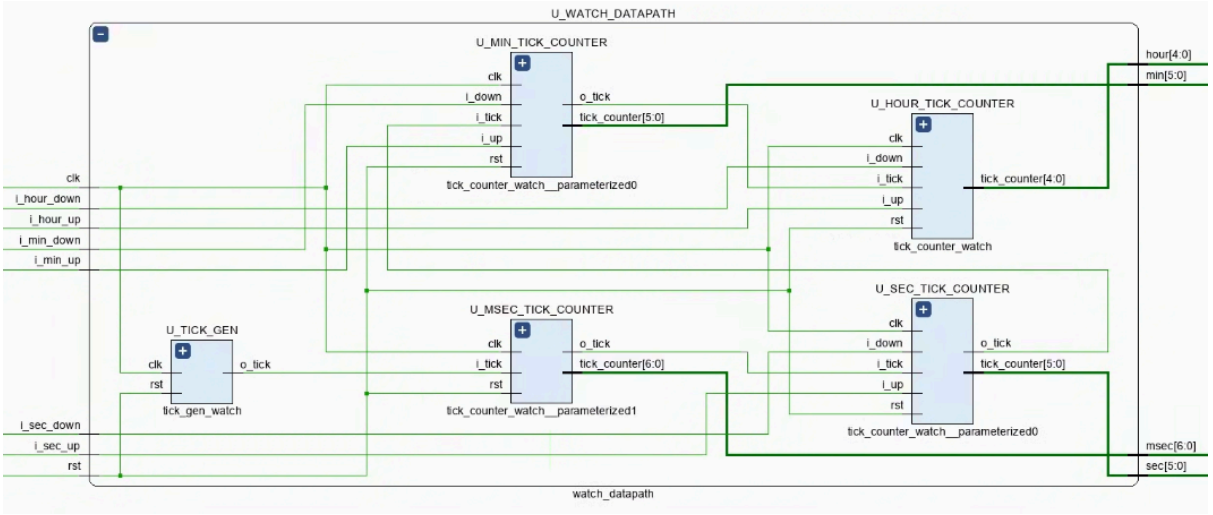
Stopwatch Datapath는 stopwatch 기능의 실제 시간 데이터 생성과 갱신을 담당하는 연산 블록으로 설계하였다. 상위 제어부에서 전달되는 run/stop, clear, mode, record, view 신호를 기반으로 msec, sec, min, hour 데이터를 생성하며, 시간 측정과 기록 기능이 하나의 데이터 경로 안에서 수행되도록 구성하였다.

기본 시간 생성은 100Hz tick generator를 기준으로 이루어진다. 100MHz system clock을 분주하여 10ms 주기의 tick을 만들고, 이를 msec counter의 입력으로 사용하였다. 이후 msec, sec, min, hour counter를 계층적으로 연결하여 하위 counter의 overflow 또는 underflow가 상위 counter의 tick으로 전달되도록 설계하였다. 이를 통해 msec → sec → min → hour 순서의 시간 증가 및 감소가 가능하도록 하였다.

또한 mode 신호에 따라 up count와 down count가 전환되도록 하였으며, clear 신호가 입력되면 모든 시간 데이터를 0으로 초기화하도록 구성하였다. run/stop 신호는 tick generator의 동작 여부를 제어하여, stopwatch가 동작 중일 때만 시간 값이 갱신되도록 하였다. 이 구조를 통해 시간 정지, 재시작, 초기화, 방향 전환 기능이 모두 datapath 내부에서 처리될 수 있도록 설계하였다.

추가적으로 record 및 view 기능을 위해 별도의 record register를 두어, 현재 시간 데이터를 저장하고 다시 불러올 수 있도록 하였다. record 신호가 입력되면 현재 시간 값이 저장되고, view 신호가 활성화되면 현재 counter 값 대신 저장된 record 값이 출력되도록 하였다. 따라서 Stopwatch Datapath는 단순 시간 계수 기능뿐 아니라 기록 및 조회 기능까지 포함하는 확장형 시간 데이터 처리 블록으로 동작한다.

- 4.4.2 Watch Datapath



Watch Datapath는 digital watch 기능의 시간 데이터 생성과 갱신을 담당하는 연산 블록으로 설계하였다. 기본적으로 msec, sec, min, hour 데이터를 계층적으로 생성하며, 자동 시간 증가 기능과 사용자 설정 기능이 하나의 datapath 안에서 함께 동작하도록 구성하였다.

시간 생성은 100Hz tick generator를 기준으로 이루어진다. 생성된 tick은 msec counter에 입력되고, msec가 한 주기를 완료하면 sec로, sec는 min으로, min은 hour로 carry가 전달되도록 설계하였다. 이를 통해 watch 기능이 별도의 외부 시간 입력 없이도 FPGA 내부에서 지속적으로 시간 데이터를 증가시키도록 하였다.

또한 watch 기능은 단순 자동 증가뿐 아니라 시, 분, 초 값을 직접 조정할 수 있도록 구성하였다. 상위 control unit에서 전달되는 hour_up, hour_down, min_up, min_down, sec_up, sec_down 신호에 따라 각 시간 단위가 직접 증가 또는 감소하도록 하였으며, 이때 각 counter는 정해진 범위를 벗어나지 않도록 순환 구조를 적용하였다. 예를 들어 min과 sec는 0~59, hour는 0~23 범위 내에서 wrap-around가 이루어지도록 설계하였다.

즉, Watch Datapath는 자동 시간 증가 구조와 수동 시간 설정 구조를 동시에 포함하는 블록으로, 기본 시계 동작과 사용자 설정 기능을 모두 지원하도록 구현하였다. 이를 통해 상위 FSM이 어떤 설정 모드를 선택하느냐에 따라 동일한 datapath에서 자동 증가 또는 직접 값 변경 동작을 수행할 수 있도록 하였다.

- 4.4.3 Time Data 통합 구조

본 시스템에서는 Stopwatch Datapath와 Watch Datapath를 각각 독립적으로 구성하되, 상위 모듈에서 두 데이터 경로를 함께 관리하는 방식으로 시간 데이터 통합 구조를 설계하였다. 즉, 두 기능은 내부적으로는 별도의 연산 블록으로 동작하지만, 출력 단계에서는 하나의 공통 인터페이스를 통해 선택적으로 사용될 수 있도록 구성하였다.

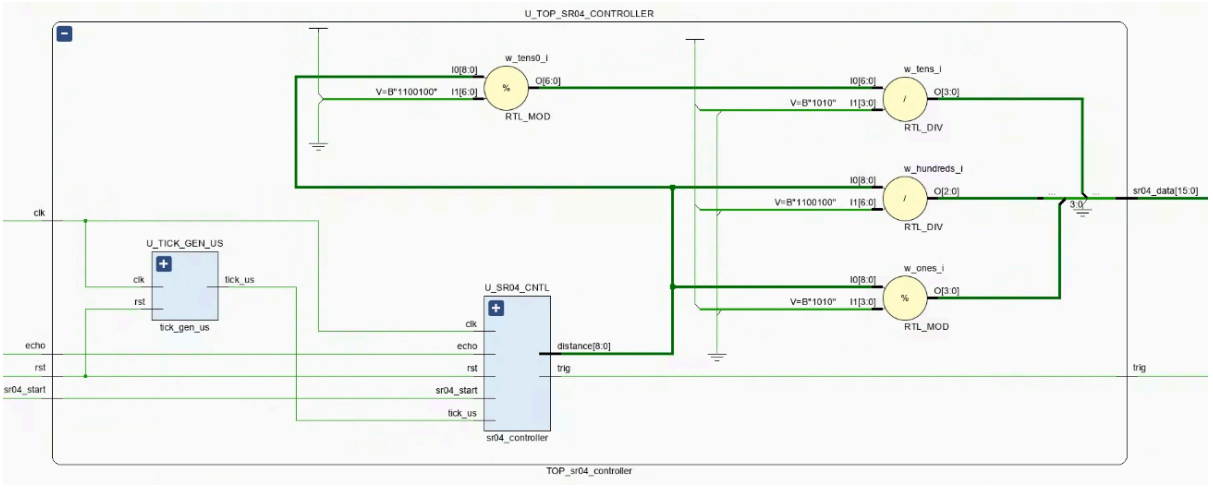
상위 시간 기능 블록에서는 stopwatch_data와 watch_data를 각각 32-bit 형태로 정리하여 관리하였다. 이 데이터는 hour, min, sec, msec 값을 하나의 버스로 묶은 형태로, 이후 UART 상태 출력이나 상위 mux 구조에서 공통적으로 활용할 수 있도록 설계하였다. 이를 통해 시간 데이터가 단순히 FND 표시용 값으로만 사용되는 것이 아니라, 외부 통신과 상위 제어 구조에서도 동일한 형식으로 재사용될 수 있도록 하였다.

또한 FND 출력 경로에서는 Stopwatch와 Watch의 msec, sec, min, hour 신호를 각각 유지한 상태에서, 스위치 및 제어 신호에 따라 현재 표시할 데이터만 선택하도록 구성하였다. 이 구조를 통해 두 기능이 동시에 내부에 존재하더라도 실제 출력은 현재 선택된 모드에 해당하는 시간 데이터만 사용되도록 하였으며, 기능 간 데이터 충돌 없이 안정적으로 동작할 수 있도록 하였다.

즉, Time Data 통합 구조는 Stopwatch와 Watch를 하나의 대형 datapath로 합치는 방식이 아니라, 두 개의 독립된 시간 데이터 경로를 유지하면서 상위 계층에서 출력 형식과 사용 목적을 통일하는 방식으로 설계되었다. 이를 통해 모듈 간 독립성을 유지하면서도, 전체 시스템 차원에서는 일관된 시간 데이터 처리 구조를 확보할 수 있었다.

4.5 센서 기능부 설계

- 4.5.1 SR04 Controller



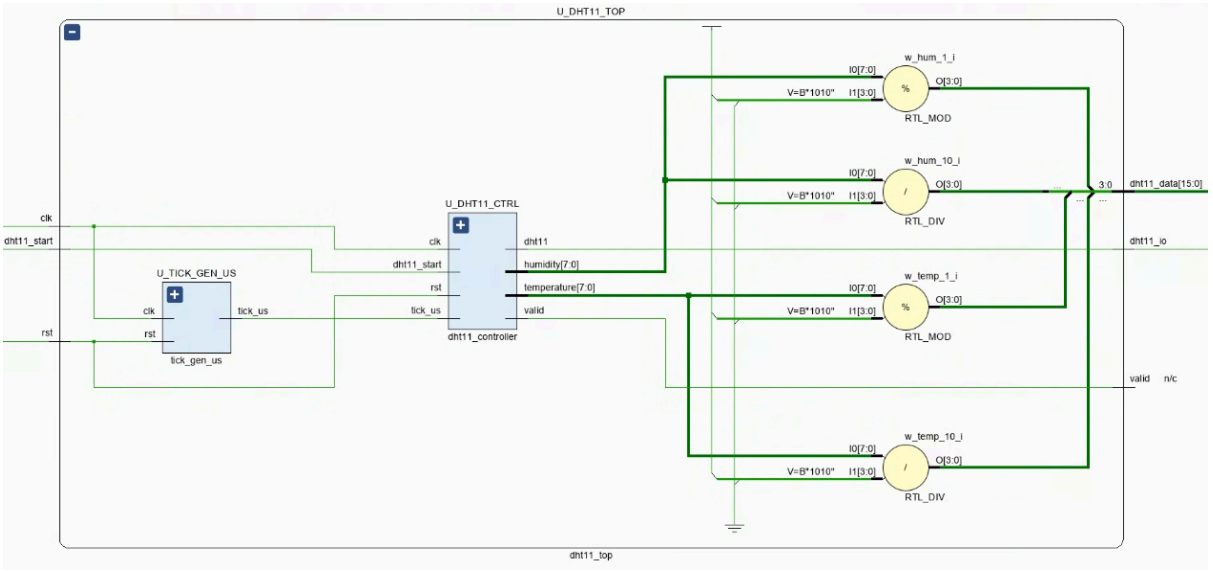
SR04 Controller은 초음파 센서의 거리 측정을 위해 IDLE, START, WAIT, RESPONSE의 4개 상태로 설계하였다. IDLE 상태에서는 측정을 대기하다가 sr04_start 신호가 입력되면 START 상태로 전이하며, 이 구간에서 trig 신호를 일정 시간 동안 High로 출력하여 센서에 측정 시작 조건을 전달하도록 하였다.

이후 WAIT 상태에서는 echo 입력이 High로 변하기를 대기하고, echo가 검출되면 RESPONSE 상태로 전이한다. RESPONSE 상태에서는 echo 신호가 High로 유지되는 시간을 counter로 계수하며, 이 값을 기반으로 초음파의 왕복 시간을 계산한 뒤 최종 거리값으로 변환하도록 구현하였다. 본 설계에서는 echo pulse width를 1us 단위로 측정하고, 이를 거리값으로 환산하는 구조를 적용하였다.

또한 측정 시작 신호가 연속적으로 입력되더라도 지나치게 자주 측정이 반복되지 않도록 시작 조건에 별도의 count 조건을 두어 동작 주기를 제한하였다. 이를 통해 센서 동작 간격을 확보하고, 불필요하게 빈번한 측정으로 인한 출력 불안정을 줄이도록 하였다.

즉, 본 RTL은 trig pulse 생성, echo 응답 대기, echo pulse width 측정, 거리 데이터 계산의 과정을 순차적으로 수행하며, FPGA 내부에서 실시간 거리 측정이 가능하도록 설계된 센서 제어 블록이다.

- 4.5.2 DHT11 Controller



DHT11 Controller는 단일 데이터 라인을 사용하는 온·습도 센서의 통신 프로토콜을 처리하기 위해 설계하였다. 평상시에는 대기 상태를 유지하다가 dht11_start 신호가 입력되면 시작 구간을 생성하고, 이후 센서의 응답 신호를 받아 데이터 수신 과정으로 진입하도록 구성하였다. 이때 FPGA가 직접 데이터를 구동하는 구간과 센서가 응답을 출력하는 구간을 분리하기 위해 tri-state 기반 inout 제어 구조를 적용하였다.

초기 시작 구간에서는 데이터 라인을 일정 시간 동안 Low로 유지하여 센서에 시작 조건을 전달하고, 이후 라인을 해제한 뒤 센서의 응답 구간을 순차적으로 검출하도록 하였다. 또한 외부 센서 입력은 system clock과 비동기 관계이므로, 내부에서는 synchronizer를 거친 신호를 사용하여 입력 안정성을 확보하였다. 이를 통해 단일선 통신에서 발생할 수 있는 오동작과 출력 충돌 가능성을 줄이도록 설계하였다.

데이터 수신 과정에서는 입력 pulse의 High 유지 시간을 계수하고, 그 길이를 기준으로 0과 1을 판별하도록 구현하였다. 이러한 과정을 반복하여 총 40bit 데이터를 순차적으로 저장하며, 최종적으로 humidity, temperature, checksum 데이터를 각각 분리해 사용할 수 있도록 구성하였다. 이후 checksum 비교를 통해 수신 데이터의 유효성을 검증하고, 정상 수신된 온도 및 습도 데이터는 FND 출력과 UART 상태 출력에 공통으로 활용할 수 있도록 설계하였다.

- 4.5.3 Sensor Data Formatting

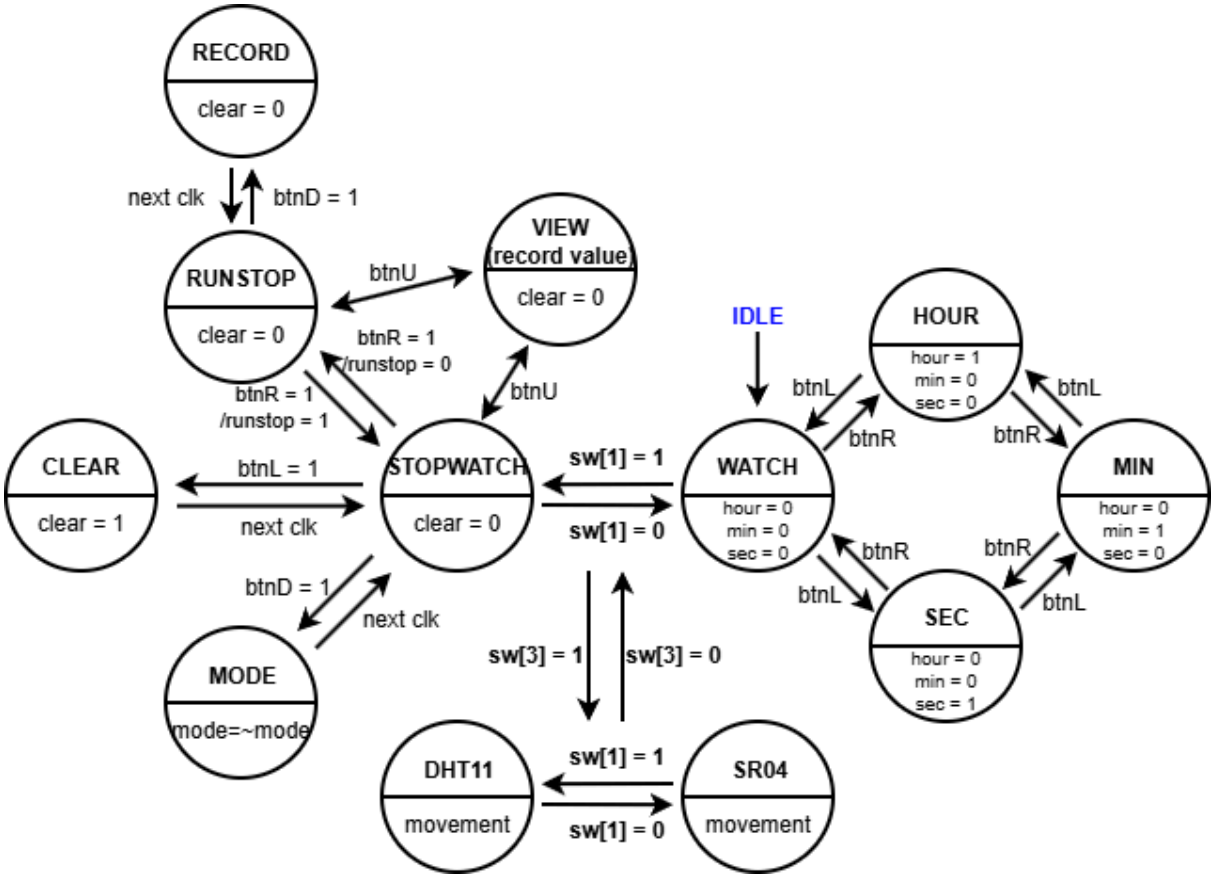
센서 데이터는 SR04와 DHT11에서 각각 서로 다른 형식으로 생성되므로, 출력부에서 공통적으로 사용할 수 있도록 별도의 데이터 정리 구조를 두었다. SR04의 경우 거리 측정 결과는 정수형 distance 값으로 생성되며, 이를 hundreds, tens, ones 자리로 분리하여 3-digit 형태의 표시 데이터로 변환하였다. 이후 상위 자리에는 0을 채워 총 16-bit 형태의 출력 데이터로 정리하고, 필요 시 leading zero를 제거하여 FND 표시 가독성을 높이도록 하였다.

DHT11의 경우 수신된 humidity와 temperature 값은 각각 8-bit 데이터로 존재하므로, 이를 10의 자리와 1의 자리로 다시 분리하여 4-digit 출력 데이터 형태로 변환하였다. 이 과정을 통해 온도 2자리와 습도 2자리를 하나의 16-bit 데이터 경로로 정리할 수 있도록 하였으며, 이후 sensor_fnd에서 별도의 추가 연산 없이 바로 출력에 사용할 수 있도록 구성하였다.

또한 상위 센서 출력 구조에서는 SR04 데이터와 DHT11 데이터를 동일한 형식의 4-digit 데이터로 맞춘 뒤, 선택 신호에 따라 하나의 공통 출력 경로로 전달하도록 설계하였다. 이를 통해 센서 종류가 달라지더라도 출력부에서는 동일한 방식으로 데이터를 처리할 수 있으며, FND 제어 구조를 단순화하고 센서 출력 경로의 재사용성을 높이도록 하였다.

4.6 제어부 설계

- 4.6.1 Control Unit FSM



Control Unit FSM은 본 시스템 전체의 동작 모드와 제어 신호를 결정하는 중앙 제어 블록으로 설계하였다. 입력으로는 실제 버튼 신호와 UART 기반 제어 신호를 함께 받아들이며, 현재 스위치 선택 상태에 따라 Stopwatch, Watch, SR04, DHT11 중 어느 기능을 활성화할지 결정하도록 구성하였다. 이를 통해 물리 버튼 입력과 UART 명령 입력이 동일한 제어 경로 안에서 처리되도록 하였다.

내부 상태는 Stopwatch 계열 상태, Watch 계열 상태, Sensor 계열 상태로 구분된다. Stopwatch 계열에서는 run/stop, clear, mode, record, view 제어 신호를 생성하고, Watch 계열에서는 hour, min, sec 선택 신호와 각 시간 단위의 up/down 제어 신호를 생성한다. 또한 Sensor 계열에서는 현재 선택된 센서에 따라 sr04_start 또는 dht11_start 신호를 활성화하여 해당 센서 블록이 동작하도록 하였다.

이 제어 구조의 특징은 단순히 상태만 전이하는 것이 아니라, 현재 상태와 입력 조건에 따라 각 기능 블록에 필요한 제어 신호를 직접 생성한다는 점이다. 예를 들어 Stopwatch 모드에서는 run/stop, clear, mode, record, view 신호가 datapath로 전달되고, Watch 모드에서는 시·분·초 조정을 위한 제어 신호가 watch datapath로 전달된다. 따라서 Control

Unit FSM은 시스템 전체에서 어떤 기능이 동작해야 하는지 결정하는 동시에, 각 블록에 필요한 제어 출력을 분배하는 역할을 수행한다.

결과적으로 본 제어부는 모드 선택, 상태 전이, 제어 신호 생성 기능을 하나로 통합한 구조이며, 시간 기능과 센서 기능이 하나의 상위 시스템 안에서 일관된 방식으로 동작할 수 있도록 하는 핵심 제어 블록이다.

- 4.6.2 버튼 입력 및 UART 입력 통합 제어

본 시스템은 실제 버튼 입력과 UART 기반 문자 명령 입력을 하나의 제어 구조 안에서 통합하여 처리하도록 설계하였다. 버튼 입력은 사용자가 FPGA 보드에서 직접 기능을 제어하기 위한 입력 경로이며, UART 입력은 외부 터미널에서 동일한 기능을 원격으로 제어하기 위한 입력 경로이다. 두 입력 방식은 입력 소스만 다를 뿐, 최종적으로는 동일한 기능 제어를 수행해야 하므로 공통 제어 경로로 연결되도록 구성하였다.

버튼 입력은 기계식 스위치 특성상 채터링이 발생할 수 있으므로, 먼저 debounce 회로를 거쳐 1-cycle pulse 형태의 안정된 신호로 변환되도록 하였다. 반면 UART 입력은 ASCII Decoder를 통해 문자 명령이 내부 제어 신호로 변환되며, 이 역시 버튼 입력과 동일한 의미를 갖는 제어 신호로 사용되도록 설계하였다. 예를 들어 R, L, U, D와 같은 UART 명령은 각각 버튼 입력과 동일한 방향 또는 기능 제어 신호로 해석되어 control unit으로 전달된다.

이후 control unit에서는 버튼 입력과 UART 입력을 각각 별도의 동작으로 구분하지 않고, 동일 조건에서 함께 처리하도록 구성하였다. 즉, 특정 기능 신호는 버튼 입력이 활성화되거나 대응되는 UART 입력이 활성화될 때 동일하게 인식되도록 설계하였다. 이를 통해 사용자는 보드의 물리적 버튼과 외부 UART 명령 중 어느 입력 방식을 사용하더라도 동일한 시스템 동작을 얻을 수 있도록 하였다.

이와 같은 통합 제어 구조를 적용함으로써 입력 방식이 달라져도 내부 제어 로직은 일관되게 유지되며, 시스템 사용 편의성과 확장성을 함께 확보할 수 있도록 하였다.

- 4.6.3 모드 전환 로직

본 시스템의 모드 전환 로직은 스위치 입력을 기준으로 Stopwatch, Watch, SR04, DHT11 기능 중 하나를 선택하도록 설계하였다. 상위 제어부에서는 sw[3]과 sw[1] 조합을 이용하여

현재 활성화할 기능을 결정하며, 이에 따라 내부 상태와 데이터 경로가 함께 전환되도록 구성하였다. 이를 통해 하나의 FPGA 시스템 안에서 시간 기능과 센서 기능을 선택적으로 사용할 수 있도록 하였다.

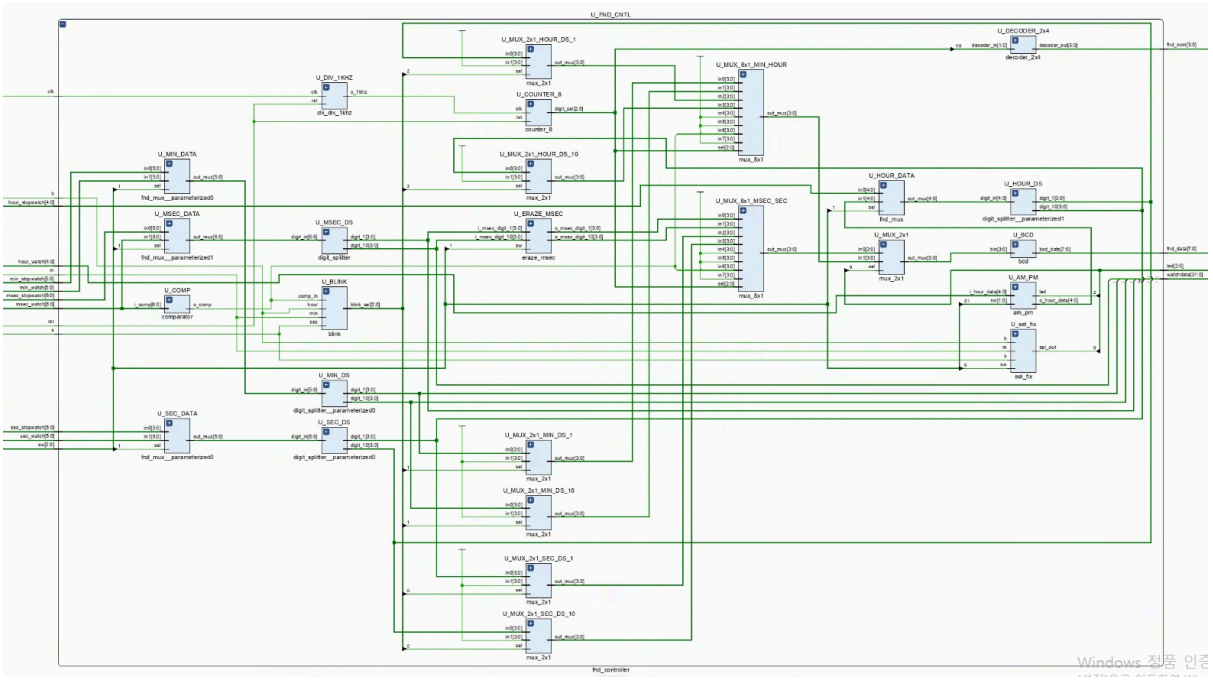
모드 전환은 단순히 출력만 바꾸는 것이 아니라, 제어부와 데이터 경로에 동시에 반영되도록 설계하였다. 즉, Stopwatch 또는 Watch가 선택된 경우에는 시간 데이터 경로와 관련 제어 신호가 활성화되고, SR04 또는 DHT11이 선택된 경우에는 해당 센서의 시작 신호와 센서 데이터 경로가 활성화된다. 따라서 현재 선택된 모드에 따라 실제로 동작하는 기능 블록과 출력 대상으로 사용되는 데이터가 함께 결정되도록 하였다.

또한 출력 단계에서도 동일한 모드 선택 기준이 적용된다. 시간 기능이 선택된 경우에는 main FND controller를 통해 Stopwatch/Watch 데이터가 표시되고, 센서 기능이 선택된 경우에는 sensor_fnd를 통해 거리값 또는 온습도 데이터가 표시된다. 동시에 UART 상태 출력 경로에서도 같은 선택 결과가 반영되어, 현재 활성화된 기능의 데이터만 ASCII 문자열 형태로 외부로 송신되도록 구성하였다.

이와 같은 구조를 통해 모드 전환 로직은 시스템 전체의 기능 선택 기준으로 동작하며, 입력 조건, 내부 제어, 데이터 처리, 출력 경로가 모두 일관된 방식으로 연동되도록 설계하였다.

4.7 출력부 설계

- 4.7.1 Main FND Controller



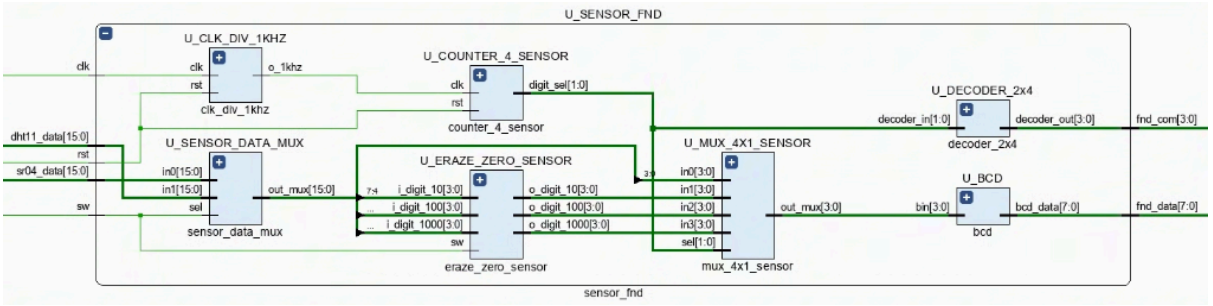
Main FND Controller는 Stopwatch와 Watch의 시간 데이터를 4-digit 7-segment 형태로 변환하여 출력하는 블록으로 설계하였다. 입력으로는 Stopwatch와 Watch에서 각각 생성된 msec, sec, min, hour 데이터와 현재 시간 설정 상태를 나타내는 제어 신호를 받으며, 선택된 모드에 따라 적절한 시간 정보를 FND에 표시하도록 구성하였다. 이를 통해 하나의 출력부에서 두 시간 기능을 공통적으로 처리할 수 있도록 하였다.

내부적으로는 먼저 Stopwatch 데이터와 Watch 데이터를 선택하는 데이터 선택 구조를 두고, 이후 각 시간 값을 digit 단위로 분리하여 출력 가능한 형태로 변환하도록 하였다. 사용자가 선택한 표시 모드에 따라 msec/sec 또는 min/hour 데이터를 출력하며, 각 digit 값은 BCD 변환 과정을 거쳐 7-segment 출력 코드로 바뀌도록 설계하였다. 이를 통해 동일한 시간 데이터라도 표시 목적에 맞게 유연하게 출력할 수 있도록 하였다.

또한 Watch 설정 모드에서는 현재 조정 중인 시간 단위를 사용자가 쉽게 구분할 수 있도록 blink 기능을 추가하였다. hour, min, sec 중 선택된 항목에 대해서만 일정 주기로 해당 digit를 깜빡이도록 구성하여, 현재 수정 대상이 무엇인지 직관적으로 확인할 수 있도록 하였다. 이와 함께 watch 모드에서는 AM/PM 표시를 위한 보조 출력도 포함하여 시간 표현의 가독성을 높이도록 설계하였다.

FND 출력은 multiplexing 방식으로 동작한다. 내부 clock divider와 digit counter를 이용하여 4개의 digit를 빠르게 순차 선택하고, 각 시점마다 해당 digit에 맞는 segment 데이터를 출력하도록 하였다. 이를 통해 제한된 FPGA 출력 핀으로 4자리 숫자를 안정적으로 표시할 수 있으며, Stopwatch와 Watch의 시간 데이터를 하나의 공통 FND 구조에서 효율적으로 출력할 수 있도록 하였다.

- 4.7.2 Sensor FND Controller



Sensor FND Controller는 SR04와 DHT11에서 생성된 센서 데이터를 4-digit 7-segment 형태로 출력하기 위한 블록으로 설계하였다. 입력으로는 SR04 거리 데이터와 DHT11 온습도 데이터를 받으며, 선택 신호에 따라 현재 활성화된 센서 데이터만 FND 출력 경로로

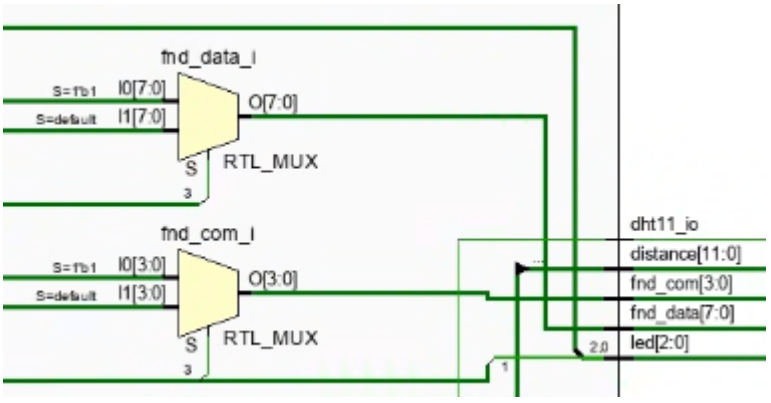
전달하도록 구성하였다. 이를 통해 서로 다른 형식의 센서 데이터를 하나의 공통 출력 구조에서 처리할 수 있도록 하였다.

내부적으로는 먼저 sensor data mux를 이용하여 SR04 데이터와 DHT11 데이터 중 하나를 선택하고, 이후 선택된 16-bit 데이터를 4개의 digit 값으로 분리하여 출력에 사용한다. SR04의 경우 거리값을 hundreds, tens, ones 형태로 표현하고, DHT11의 경우 temperature 2자리와 humidity 2자리를 그대로 4-digit 형태로 표시하도록 구성하였다. 이렇게 각 센서 데이터를 동일한 4자리 출력 형식으로 맞춤으로써 상위 출력 구조를 단순화하였다.

또한 SR04 모드에서는 실제 거리값이 3자리 이하인 경우가 많으므로, 상위 자리의 불필요한 0이 표시되지 않도록 leading zero 제거 로직을 추가하였다. 이를 통해 거리 데이터의 가독성을 높이고, 센서 출력이 보다 직관적으로 보이도록 하였다. 반면 DHT11 모드에서는 온도와 습도를 모두 유효한 4자리 데이터로 사용하므로 별도의 zero 제거 없이 그대로 출력하도록 설계하였다.

FND 출력 방식은 main FND controller와 동일하게 multiplexing 구조를 사용하였다. 내부 clock divider와 digit counter를 이용하여 4개의 digit를 빠르게 순차 선택하고, 현재 선택된 digit에 대응하는 segment 데이터를 출력하도록 구성하였다. 이를 통해 SR04와 DHT11 센서 데이터를 하나의 공통 FND 자원으로 안정적으로 표시할 수 있도록 하였다.

- 4.7.3 FND 출력 선택 구조



본 시스템은 하나의 FND 자원을 시간 데이터와 센서 데이터가 공통으로 사용하도록 설계하였으므로, 최종 출력 단계에서 표시 대상을 선택하는 구조가 필요하다. 이를 위해 main FND controller와 sensor FND controller를 각각 독립적으로 구성한 뒤, 상위 모듈에서 스위치 입력을 기준으로 두 출력 경로 중 하나를 선택하도록 하였다.

시간 기능이 선택된 경우에는 Stopwatch/Watch 데이터가 main FND controller를 통해

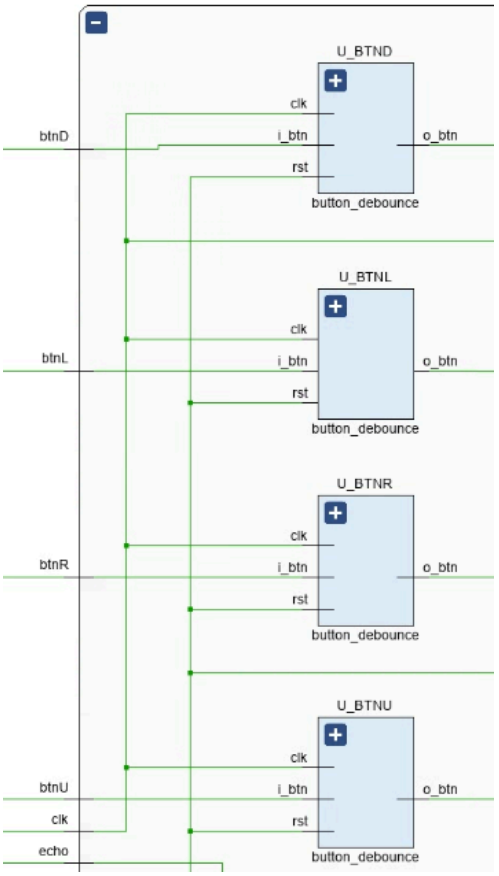
출력되고, 센서 기능이 선택된 경우에는 SR04 또는 DHT11 데이터가 sensor FND controller를 통해 출력된다. 상위 모듈에서는 sw[3] 신호를 기준으로 두 출력 경로를 구분하여, 현재 선택된 기능에 해당하는 fnd_com과 fnd_data만 최종 출력 포트로 전달하도록 구성하였다.

이 구조의 장점은 시간 기능과 센서 기능의 출력부를 서로 분리된 상태로 유지할 수 있다는 점이다. 각 출력 블록은 자신에게 필요한 데이터 형식과 제어 방식을 독립적으로 처리하고, 최종 단계에서만 선택적으로 연결되므로 설계가 단순해지고 디버깅도 용이해진다. 또한 하나의 FND 자원을 공유하면서도 기능별 출력 특성을 그대로 유지할 수 있어 자원 활용 측면에서도 효율적이다.

즉, FND 출력 선택 구조는 시간 데이터 출력과 센서 데이터 출력을 하나의 시스템 안에서 통합하면서, 현재 모드에 따라 적절한 표시 결과만 사용자에게 제공하도록 하는 최종 출력 인터페이스 역할을 수행한다.

4.8 입력 안정화 및 보조 회로

- 4.8.1 Button Debounce

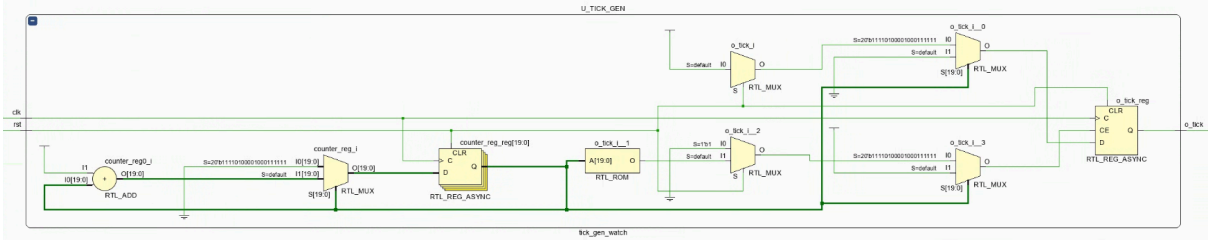


Button Debounce 회로는 기계식 버튼 입력에서 발생하는 채터링 현상을 제거하고, 이를 1-cycle pulse 형태의 안정된 제어 신호로 변환하기 위해 설계하였다. 기계식 버튼은 한 번 눌렀을 때도 매우 짧은 시간 동안 여러 번 On/Off가 반복되는 특성이 있으므로, 이를 그대로 FSM 입력에 사용할 경우 하나의 입력이 여러 번 인식되어 의도하지 않은 상태 전이가 발생할 수 있다. 본 시스템에서는 이러한 문제를 방지하기 위해 버튼 입력을 직접 사용하지 않고, 반드시 debounce 회로를 거친 뒤 제어부로 전달하도록 구성하였다.

내부적으로는 먼저 system clock을 분주하여 상대적으로 낮은 주파수의 sampling clock을 생성하고, 이 sampling clock을 기준으로 버튼 입력을 일정 주기마다 관찰하도록 하였다. 이후 입력값을 shift register에 순차적으로 저장하여, 여러 샘플 구간 동안 동일한 값이 유지되는 경우에만 안정된 입력으로 판단하도록 설계하였다. 즉, 순간적인 튐 현상은 무시하고 일정 시간 이상 유지된 신호만 유효 입력으로 인정하는 구조이다.

안정화된 입력은 이후 edge detect 과정을 거쳐 1-cycle pulse로 변환된다. 이 과정을 통해 버튼이 눌린 상태가 일정 시간 유지되더라도 control unit에서는 한 번의 이벤트로만 인식할 수 있도록 하였다. 결과적으로 debounce 회로는 버튼 입력의 물리적 불안정성을 제거하고, FSM 및 datapath가 신뢰성 있게 동작할 수 있도록 하는 입력 안정화 블록의 역할을 수행한다.

- 4.8.2 Tick Generator 및 보조 타이밍 회로



본 시스템은 UART 통신, 시간 데이터 생성, 센서 제어, FND 표시와 같이 서로 다른 시간 기준을 필요로 하는 기능들을 포함하므로, 각 기능에 맞는 보조 timing 회로를 함께 설계하였다. 이러한 보조 회로들은 모두 100 MHz system clock을 기준으로 동작하며, 필요한 주기의 tick 또는 분주 clock을 생성하여 각 기능 블록이 안정적으로 동작할 수 있도록 한다.

시간 기능부에서는 Stopwatch와 Watch의 시간 생성을 위해 100 Hz tick generator를 사용하였다. 이 신호는 10 ms 주기의 기준 timing으로 사용되며, msec counter의 입력으로 전달된 뒤 sec, min, hour까지 계층적으로 확장되도록 구성하였다. 이를 통해 별도의 외부 시간원 없이도 FPGA 내부에서 일정한 시간 증가 구조를 구현할 수 있도록 하였다.

센서 기능부에서는 SR04와 DHT11의 timing 제어를 위해 1 us 단위의 tick generator를 사용하였다. SR04는 trig pulse 생성과 echo pulse width 측정에 정확한 마이크로초 단위 timing이 필요하고, DHT11 역시 시작 신호 유지 시간과 응답 pulse width 판별이 마이크로초 단위로 이루어지므로, 동일한 us tick을 두 센서 제어부에서 공통 기준으로 사용하도록 설계하였다. 이를 통해 센서 프로토콜을 RTL 수준에서 보다 명확하게 구현할 수 있었다.

출력부에서는 FND multiplexing을 위해 1 kHz 분주 회로를 사용하였다. 이 신호는 digit counter의 입력으로 사용되어 4개의 digit를 빠르게 순차 선택하도록 하며, 사용자가 보기에는 모든 자리가 동시에 점등된 것처럼 보이도록 한다. 결과적으로 Tick Generator 및 보조 timing 회로는 각 기능 블록이 요구하는 시간 단위를 FPGA 내부에서 생성하는 기반 회로이며, 시스템 전체의 동작 안정성과 timing 일관성을 확보하는 데 중요한 역할을 수행한다.

5. 시뮬레이션 및 검증 (Simulation & Verification)

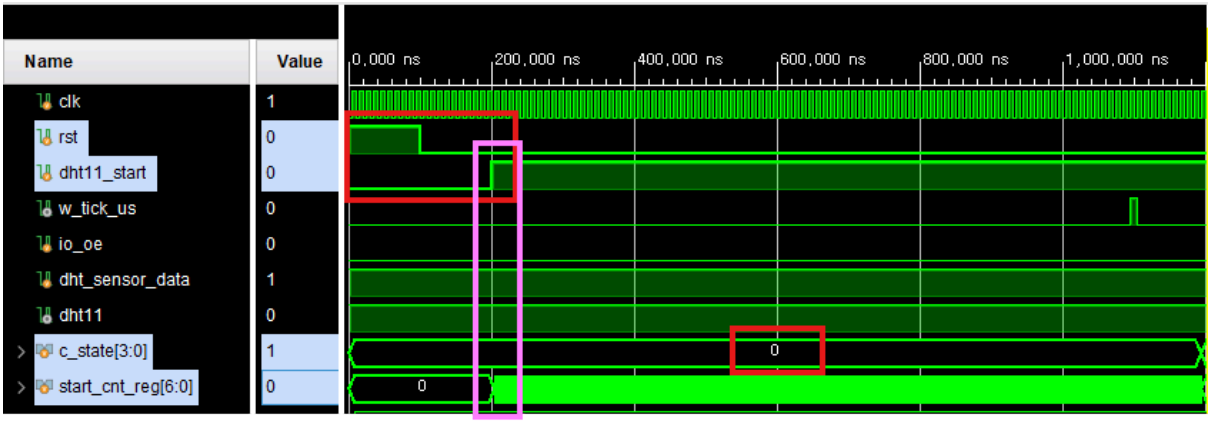
5.1 시뮬레이션 시나리오

시나리오 번호	검증 대상	입력 조건	기대 동작
시나리오 1	DHT11 Controller	reset 해제 후 dht11_start 인가, 이후 가상의 DHT11 응답 신호(80us low + 80us high)와 40bit 데이터 스트림 입력	컨트롤러가 시작 신호를 생성한 뒤 센서 응답을 인식하고, 40bit 데이터를 순차적으로 수신하여 humidity=60, temperature=25, valid=1을 출력해야 한다.
시나리오 2	SR04 Controller	reset 해제 후 sr04_start 인가, trig 종료 후 일정 지연 뒤 echo를 1160us 동안 high로 유지	컨트롤러가 trig pulse를 정상적으로 출력하고, echo pulse width를 측정하여 약 20cm의 거리값을 계산해야 한다.
시나리오 3	SR04 Controller	sr04_start 인가 후 echo를 입력하지 않음	측정 시작 이후 echo가 들어오지 않는 경우 거리값이 갱신되지 않고, 응답 대기 상태가 유지되는지 확인한다.

시나리오 4	top_fnd	sw=4'b0000	watch 모드에서 msec/sec 데이터가 main FND 경로로 출력되어야 한다.
시나리오 5	top_fnd	sw=4'b0001	watch 모드에서 min/hour 데이터가 main FND 경로로 출력되어야 한다.
시나리오 6	top_fnd	sw=4'b0010	stopwatch 모드가 선택되고, stopwatch 시간 데이터가 main FND 경로로 출력되어야 한다.
시나리오 7	top_fnd	sw=4'b1000	sensor FND 경로가 선택되고, SR04 거리 데이터가 FND에 표시되어야 한다.
시나리오 8	top_fnd	sw=4'b1010	sensor FND 경로가 선택되고, DHT11 온습도 데이터가 FND에 표시되어야 한다.
시나리오 9	top_fnd	btnR, btnL를 순차 입력하여 WATCH → HOUR → MIN → SEC 및 역방향 전이 수행	watch 설정 상태가 정상적으로 전이하고, 선택된 시간 단위에 해당하는 blink 표시가 바르게 동작해야 한다.
시나리오 10	top_uart	UART 입력으로 R, L, U, D, M, S 문자 순차 인가	RX 경로에서 ASCII 문자가 정상 수신되고, ASCII Decoder가 각 문자에 대응하는 내부 제어 신호(uart_R, uart_L, uart_U, uart_D, uart_M, uart_S)를 정확히 생성해야 한다.
시나리오 11	top_uart	select=00, data=시간 데이터, 이후 S 문자 입력	현재 선택된 watch 데이터가 ASCII 문자열 형식으로 변환되어 TX FIFO 및 UART TX 경로로 전달되어야 한다.
시나리오 12	top_uart	select=01, 10, 11로 순차 변경 후 각각 S 문자 입력	stopwatch, SR04, DHT11 데이터가 각 모드에 맞는 ASCII 문자열 형식으로 변환되어 UART TX로 순차 송신되어야 한다.

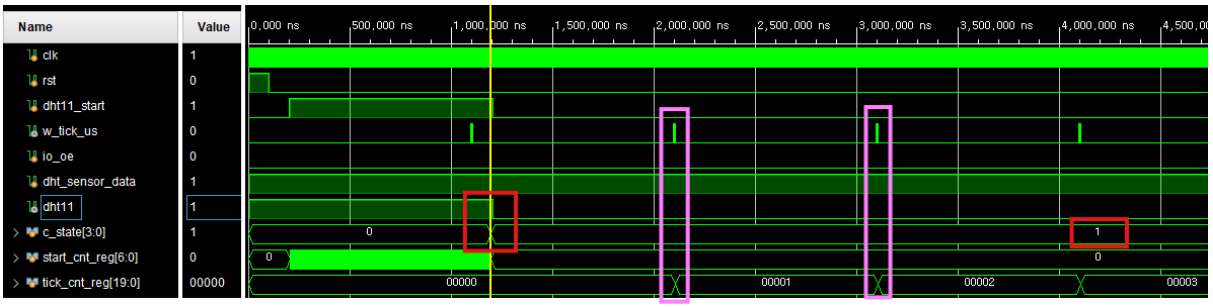
5.2 Waveform 결과

- 5.2.1 DHT11 test



case 1. rst=1일 때 c_state = IDLE, rst = 0 이후 dht11_start = 1, start_cnt_reg 증가

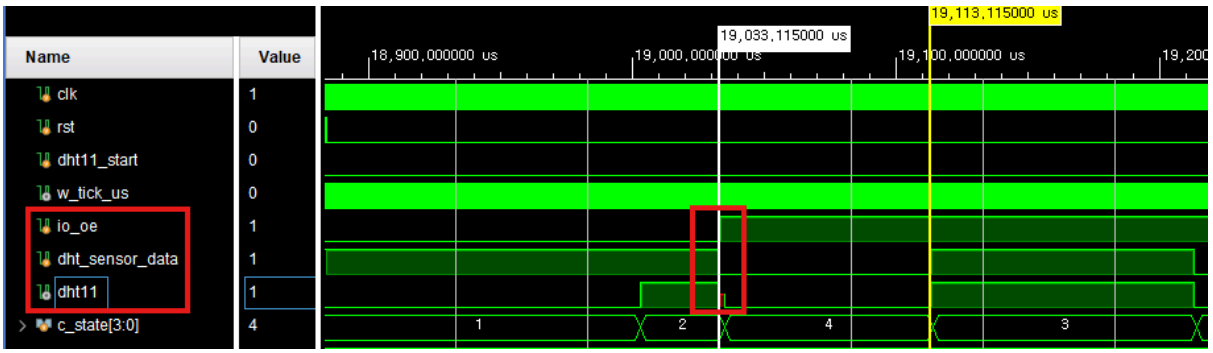
rst=1 구간에서는 컨트롤러가 초기화 상태를 유지하므로 c_state는 IDLE이고, start_cnt_reg도 0으로 유지된다.
rst가 0으로 해제된 뒤 dht11_start=1이 입력되면, FSM은 바로 전이하지 않고 IDLE 상태에서 start_cnt_reg를 증가시키며 시작 조건을 계수한다.
즉, 본 파형은 DHT11 제어기가 리셋 해제 후 내부 시작 타이밍을 먼저 맞춘 뒤 다음 상태로 전이하도록 설계되었음을 보여준다.



case 2. START 진입 후 dht11 = 0 출력, tick_cnt_reg 증가

start 조건이 만족되면 FSM은 IDLE에서 START 상태로 전이하며, c_state가 1로 변경되는 것을 확인할 수 있다.
이 구간에서는 FPGA가 DHT11 시작 신호를 생성하기 위해 dht11 라인을 low로 구동하고, 이에 따라 dht11 값이 0으로 내려간다.

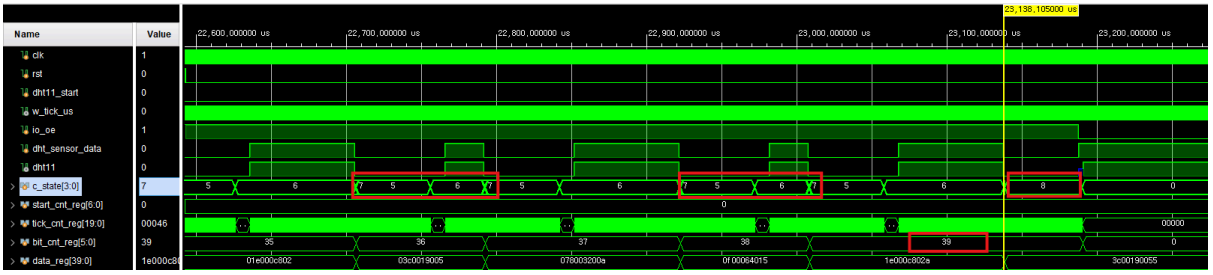
동시에 tick_cnt_reg가 증가하면서 START 상태 유지 시간이 계수되며, 이후 정해진 시간 조건을 만족하면 다음 상태로 전이하게 된다.





case 3. DHT11 response 구간, dht11 = 0 유지

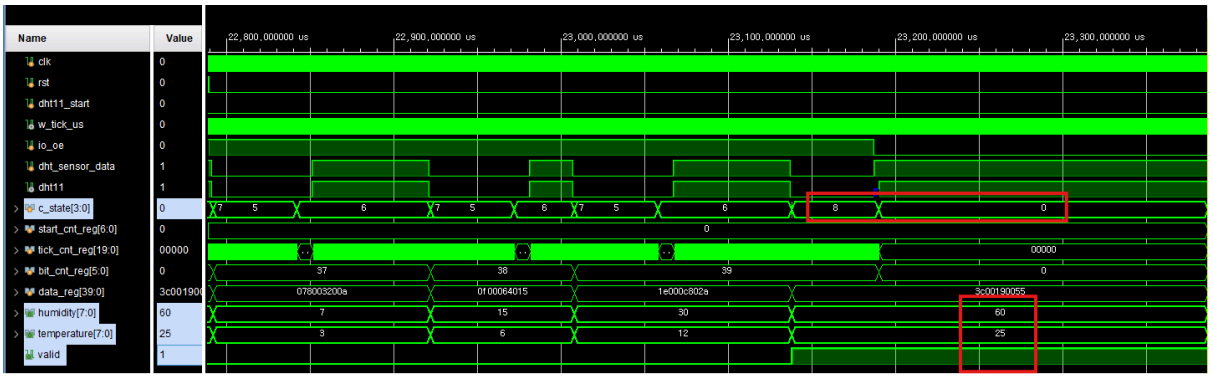
START 이후 응답 구간에 진입하면 DHT11 센서는 초기 응답 신호로 데이터 라인을 low로 유지하며, 파형에서 dht_sensor_data = 0, dht11 = 0 상태가 약 80us 동안 지속되는 것을 확인할 수 있다.
이 구간은 센서가 FPGA의 시작 신호를 정상적으로 인식한 뒤 응답을 반환하는 단계이며, 이후 FSM은 다음 동기화 구간으로 전이한다.
즉, 본 파형은 DHT11의 초기 response low 구간이 정상적으로 검출되고 있음을 보여준다.



case 4. 40bit 데이터 수신 후 STOP 전이

데이터 수신 구간에서는 c_state가 DATA_SYNC(5) → DATA_COUNT(6) → DATA_DECISION(7) 순서로 반복되며, 각 비트가 순차적으로 판별된다.
이 과정에서 bit_cnt_reg가 1씩 증가하고, data_reg에는 판별된 0/1 값이 누적되면서 40bit

데이터가 점차 채워진다.
마지막 비트 수신 후 bit_cnt_reg가 39에 도달하면 FSM은 STOP 상태로 전이하며, 전체 수신이 완료되었음을 확인할 수 있다.

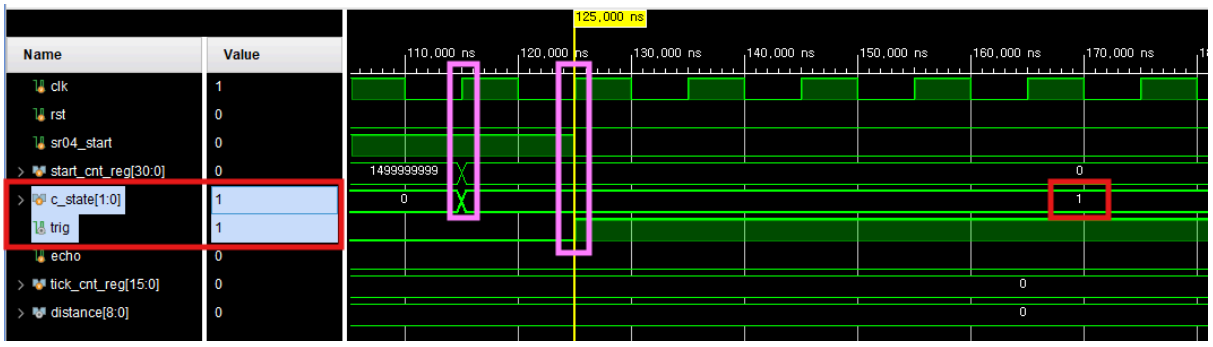



```
=====================================
TEST PASS
humidity    = 60
temperature = 25
valid       = 1
=====================================
```

case 5. 수신 종료 후 STOP → IDLE 전이, humidity=60, temperature=25, valid=1

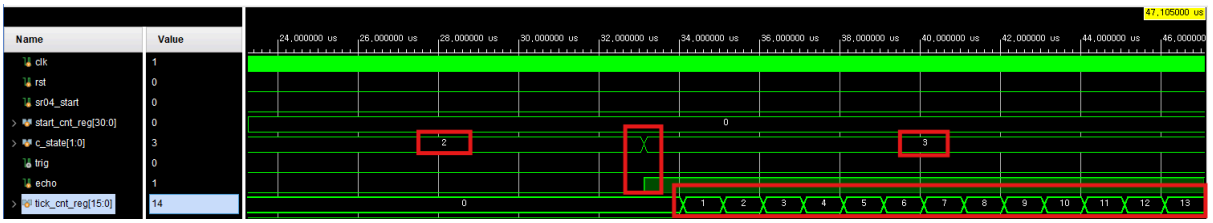
마지막 비트 수신이 완료되면 FSM은 STOP 상태를 거친 뒤 다시 IDLE 상태로 복귀하며, 전체 DHT11 수신 과정이 정상적으로 종료된다.
이때 data_reg에 저장된 40bit 데이터로부터 humidity=60, temperature=25가 정상적으로 분리되어 출력되는 것을 확인할 수 있다.
또한 checksum 비교 결과 valid=1이 유지되므로, 최종 수신 데이터가 유효한 값임을 확인할 수 있다.

- 5.2.2 SR04 test



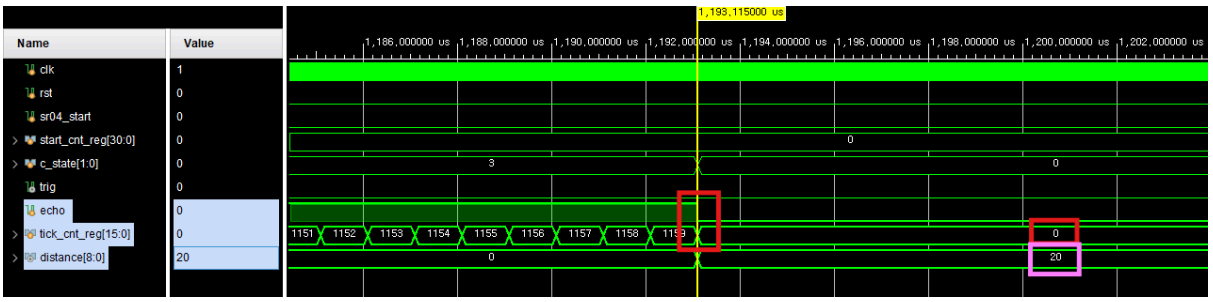
case 1. START 상태에서 trig=1 출력 확인

sr04_start 입력 후 FSM이 START 상태(c_state=1)로 전이하면, trig 신호가 실제로 1로 출력되는 것을 확인할 수 있다.
이는 SR04 센서에 측정 시작용 trigger pulse가 정상적으로 인가되고 있음을 의미한다.
즉, 본 파형은 START 구간에서 trig 출력이 의도한 대로 활성화되는지를 검증한 결과이다.



case 2. echo=1 검출 후 RESPONSE 상태에서 tick_cnt_reg 증가 확인

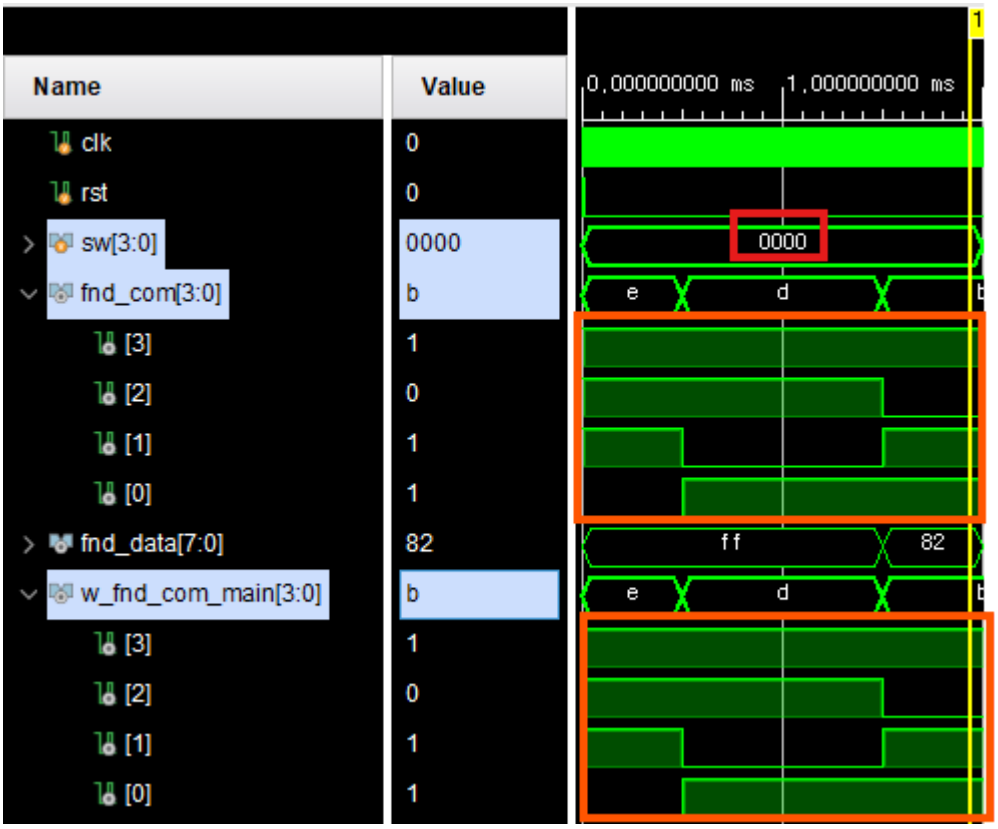
WAIT 상태(c_state=2)에서 echo가 1로 입력되면 FSM이 RESPONSE 상태(c_state=3)로 전이하는 것을 확인할 수 있다.
이후 RESPONSE 구간에서는 echo가 High로 유지되는 동안 tick_cnt_reg가 1씩 증가하며 echo pulse width를 계수한다.
즉, 본 파형은 거리 계산을 위한 시간 측정이 RESPONSE 상태에서 정상적으로 수행되고 있음을 보여준다.



case 3. echo 종료 후 distance=20 저장 확인

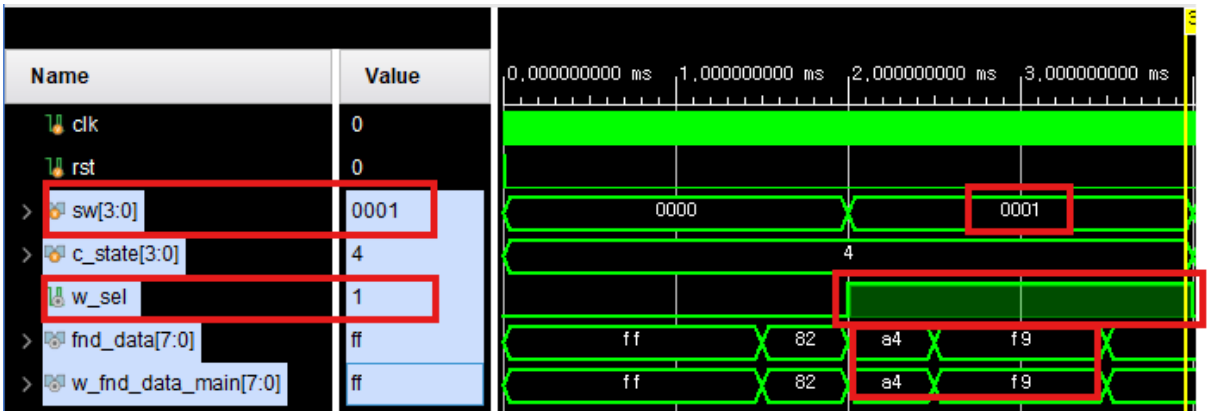
RESPONSE 상태에서 echo가 0으로 내려가면 tick_cnt_reg 값이 더 이상 증가하지 않고, 측정이 종료된다.
이때 testbench에서 echo를 1160us 동안 High로 유지하도록 설정했으므로, 거리 계산식에 따라 distance = 1160 / 58 = 20이 저장된다.
즉, 본 파형은 echo pulse width 측정 결과가 정상적으로 거리값 20으로 변환되어 출력됨을 보여준다.

- 5.2.3 Top_FND simulation result



case 1. sw=0000일 때 main FND 출력 선택 확인

sw=0000인 경우 최종 출력 선택 조건에 따라 fnd_com과 fnd_data는 sensor FND가 아니라 main FND 출력값인 w_fnd_com_main, w_fnd_data_main을 그대로 따르게 된다.
파형에서도 실제 fnd_com이 w_fnd_com_main과 동일하게 동작하고, fnd_data 역시 main 경로의 값만 반영되는 것을 확인할 수 있다.
즉, 이 구간에서는 sensor 쪽 출력값이 내부에서 어떤 값을 가지더라도 최종 FND 출력에는 반영되지 않음을 보여준다.

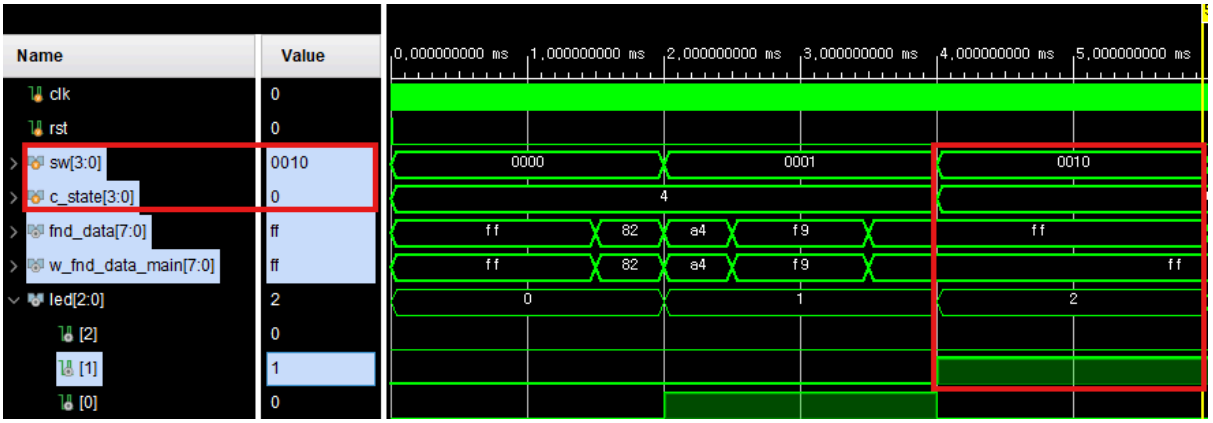


case 2. sw=0001일 때 min/hour 표시 경로 선택 확인

sw가 0001로 바뀌면 watch 모드(c_state=4)에서 w_sel이 1로 설정되어 msec/sec가 아니라 min/hour 표시 경로가 선택된다.

이때 최종 fnd_data와 내부 main 출력인 w_fnd_data_main이 동일하게 나타나므로, 선택된 main FND 데이터가 그대로 최종 출력으로 전달되는 것을 확인할 수 있다.

즉, 본 파형은 sw=0001 조건에서 min/hour 표시 모드가 정상적으로 선택되고, 해당 데이터가 FND 출력에 반영됨을 보여준다.

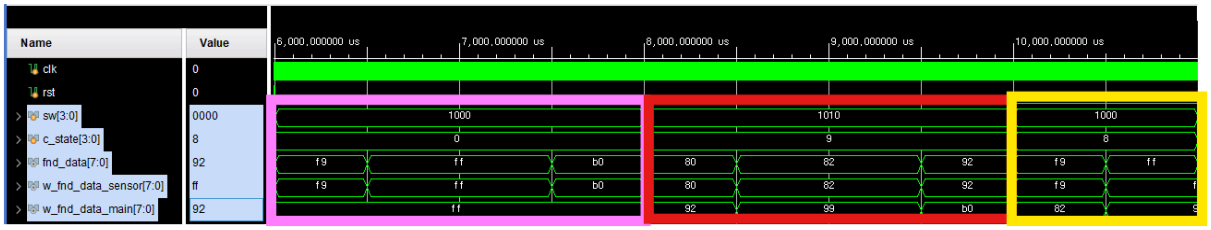


case 3. sw=0010일 때 STOPWATCH 모드 선택, led[1]=1 확인

sw가 0010으로 설정되면 c_state가 0으로 유지되어 시스템이 STOPWATCH 모드로 동작하는 것을 확인할 수 있다.

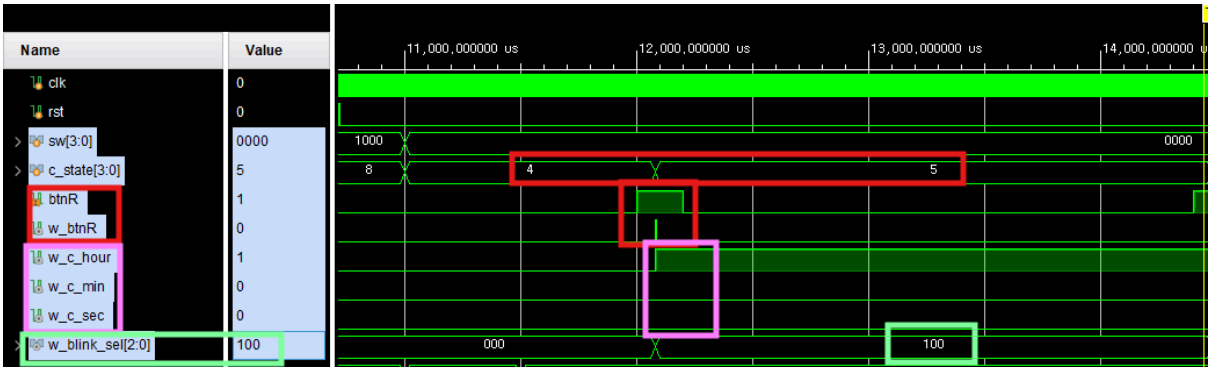
또한 led[1]은 설계에서 sw[1]에 직접 연결되어 있으므로, sw[1]=1인 이 구간에서 led[1]도 1로 출력된다.

이때 fnd_data와 w_fnd_data_main의 값이 함께 변하는 것으로 보아, stopwatch 경로의 main FND 데이터가 최종 출력에 정상적으로 반영되고 있음을 확인할 수 있다.



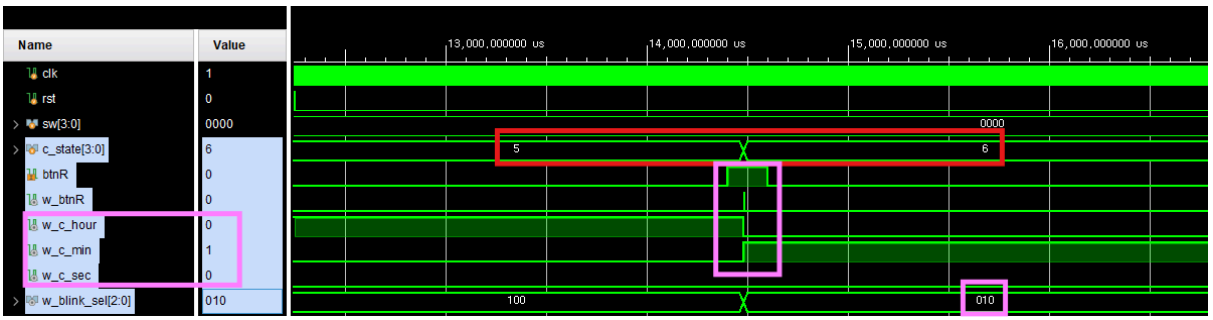
case 4. sw=1000 → 1010 → 1000 전환에 따른 c_state 변화 및 sensor FND 출력 확인

sw가 1000으로 바뀐 직후에는 c_state가 아직 0(STOPWATCH)로 남아 있는데, 이는 FSM 상태가 posedge clk에서 갱신되기 때문이며 이 구간에서도 출력 선택은 sw[3]=1에 의해 이미 sensor FND 경로를 따른다.
이후 sw=1010이 되면 c_state가 9(DHT11)로 전이하고, sensor FND는 DHT11 데이터 출력값을 반영하므로 앞 구간과 다른 fnd_data가 나타난다.
다시 sw=1000으로 변경되면 c_state가 8(SR04)로 전이하며, sensor FND 출력도 DHT11 값에서 SR04 값으로 다시 바뀌는 것을 확인할 수 있다.



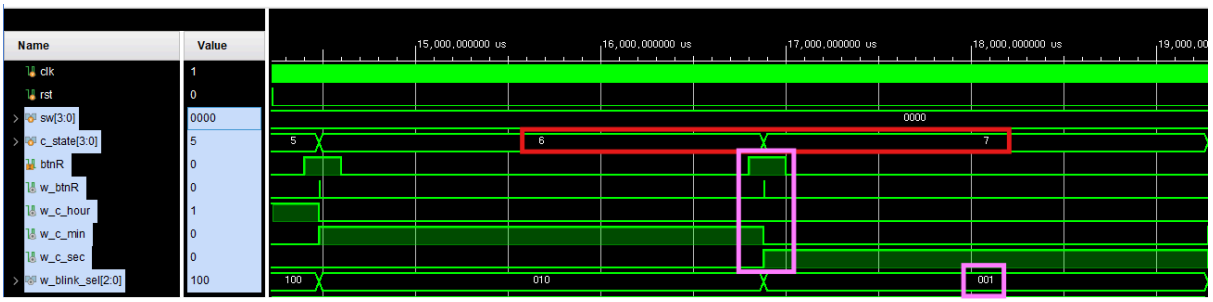
case 5. WATCH(4) → HOUR(5) 상태 전이

btnR 입력이 debounce와 동기화 과정을 거쳐 w_btnR 펄스로 인식되면서, c_state가 WATCH(4)에서 HOUR(5)로 전이하는 것을 확인할 수 있다.
HOUR 상태에 진입하면 시간 설정 대상으로 hour만 선택되므로 w_c_hour=1, w_c_min=0, w_c_sec=0이 된다.
또한 w_blink_sel=100으로 출력되어, FND에서 hour 자리가 blink되며 현재 시(hour) 수정 모드임을 보여준다.



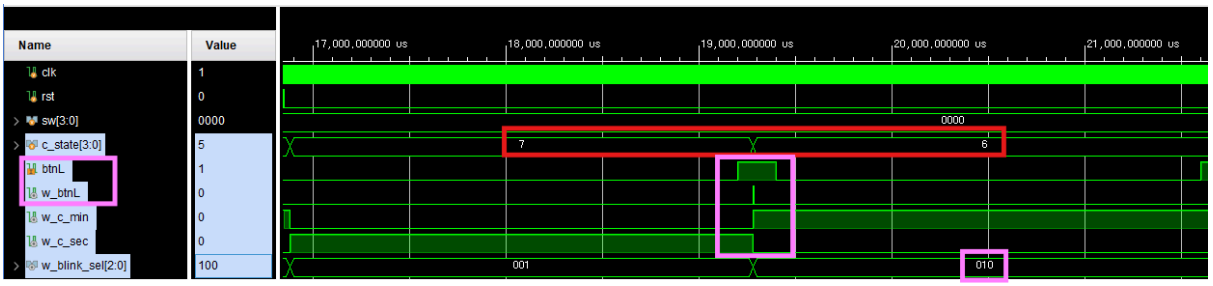
case 7. HOUR(5) → MIN(6) 상태 전이

HOUR 설정 상태에서 다음 입력이 들어오면 c_state가 5에서 6으로 바뀌며, 분(min) 수정 모드로 전이하는 것을 확인할 수 있다.
이때 제어 신호도 w_c_hour=0, w_c_min=1, w_c_sec=0으로 변경되어 수정 대상이 hour에서 min으로 넘어간다.
또한 w_blink_sel이 100에서 010으로 바뀌어, FND의 blink 위치도 hour 자리에서 min 자리로 이동함을 확인할 수 있다.



case 8. MIN(6) → SEC(7) 상태 전이

MIN 설정 상태에서 다음 입력이 들어오면 c_state가 6에서 7로 바뀌며, 초(sec) 수정 모드로 전이하는 것을 확인할 수 있다.
이때 제어 신호는 w_c_hour=0, w_c_min=0, w_c_sec=1로 변경되어 수정 대상이 min에서 sec로 넘어간다.
또한 w_blink_sel이 010에서 001로 바뀌어, FND의 blink 위치도 min 자리에서 sec 자리로 이동함을 확인할 수 있다.



case 9. SEC(7) → MIN(6) 상태 전이

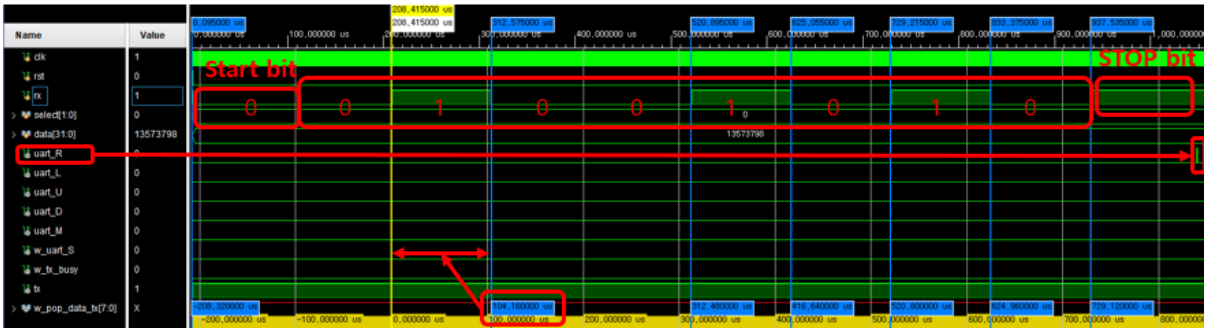
SEC 설정 상태에서 btnL 입력이 인가되면 c_state가 7에서 6으로 바뀌며, 다시 분(min) 수정 모드로 진입하는 것을 확인할 수 있다.
이때 blink 선택 신호도 001에서 010으로 변경되어, FND의 blink 위치가 sec 자리에서 min 자리로 이동한다.
즉, btnL 입력을 통해 이전 설정 단계로 정상 복귀하며, 표시 역시 현재 수정 대상에 맞게 함께 변경됨을 확인할 수 있다.

- 5.2.4 uart_rx 출력

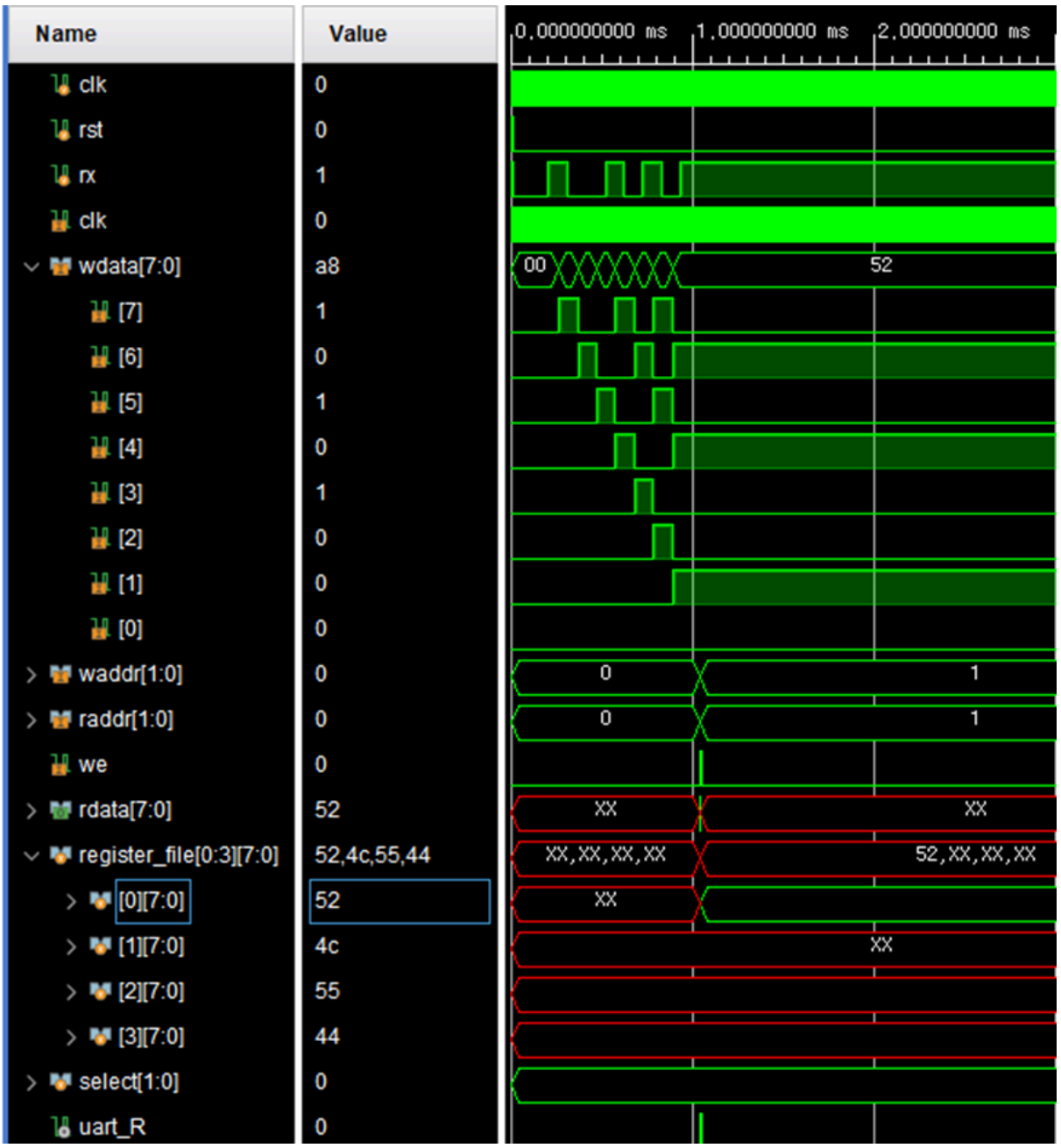
문자	Hex	Binary	LSB→MSB
R	52	01010010	01001010
L	4C	01001100	00110010
U	55	01010101	10101010
D	44	01000100	00100010
M	4D	01001101	10110010
S	53	01010011	11001010

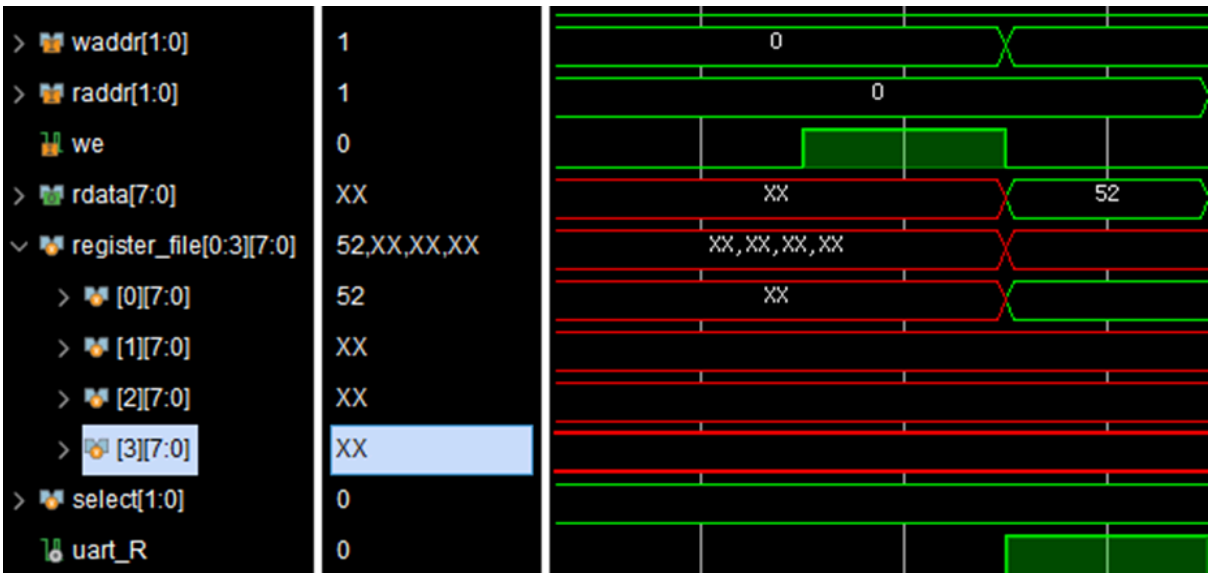
<R,L,U,D,M,S> 의 Hex,Binary 진리표

case1. uart_R 출력



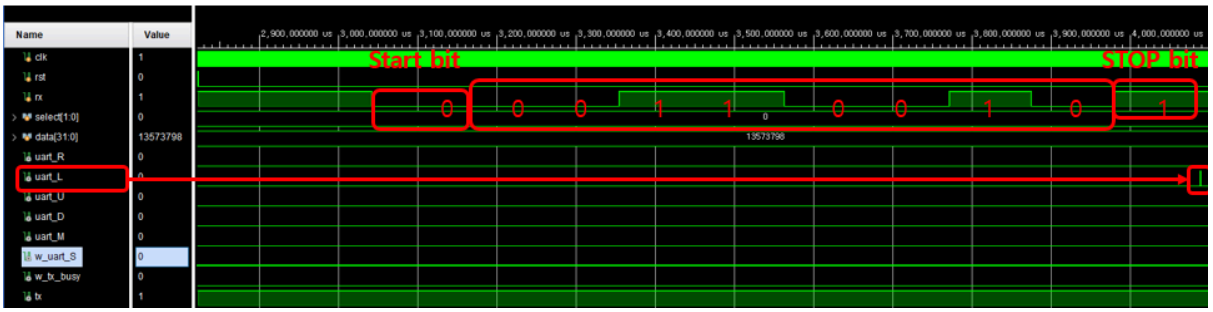
select에 상관없이 9600hz의 baud tick 에 맞추어 comportmaster로 rx를 입력하였을때 uart의 값에 따라 ascii decoder에서 uart 출력이 발생한다.


Name
Value
0.000000000 ms
1.000000000 ms
2.000000000 ms
clk
0
rst
0
rx
1
clk
0
wdata[7:0]
a8
00
52
[7]
1
[6]
0
[5]
1
[4]
0
[3]
1
[2]
0
[1]
0
[0]
0
waddr[1:0]
0
0
1
raddr[1:0]
0
0
1
we
0
rdata[7:0]
52
XX
XX
register_file[0:3][7:0]
52,4c,55,44
XX,XX,XX,XX
52,XX,XX,XX
[0][7:0]
52
XX
[1][7:0]
4c
XX
[2][7:0]
55
[3][7:0]
44
select[1:0]
0
uart_R
0

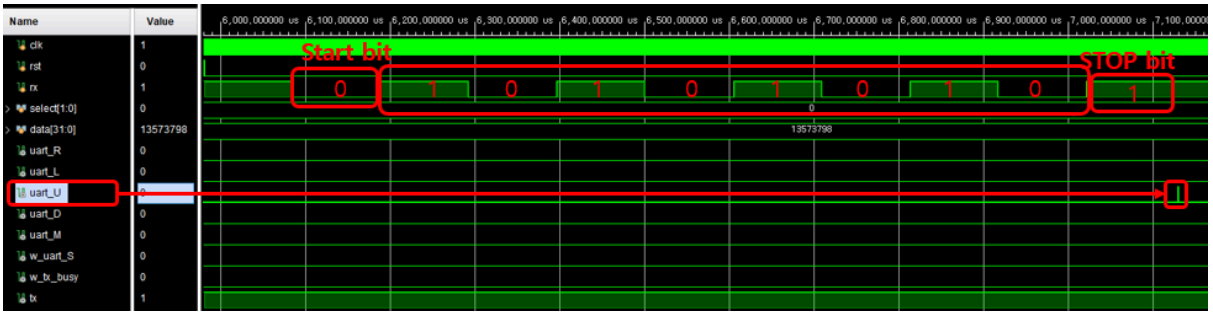


이때 rx_fifo 에 wdata를 8비트를 입력 받은 후 we가 1이 되고 1clk 후 fifo_rx의 register_file[waddr]에 rx 값이 입력 되고 we=1 인 경우 이 값이 ascii decoder로 다음 clk 에 출력 되어 uart 출력이 발생한다.

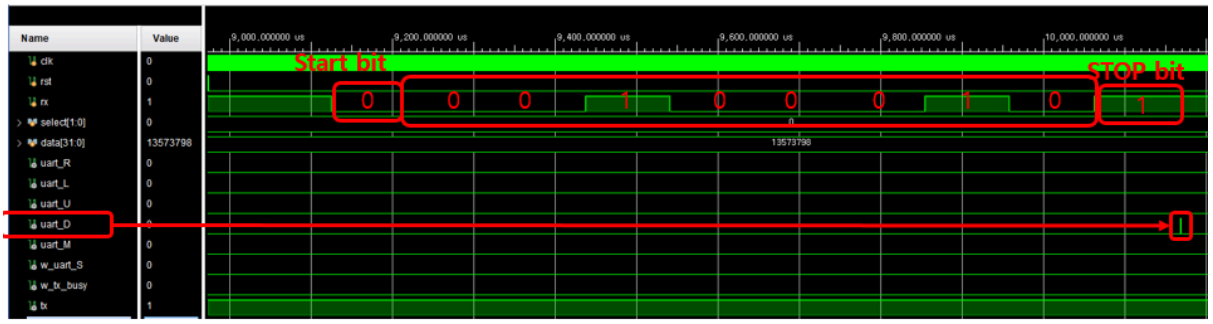
case2. uart_L 출력



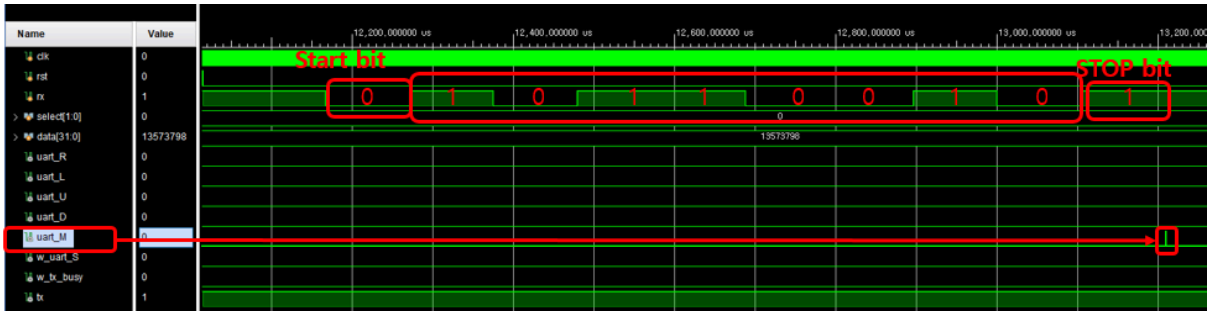
case3. uart_U 출력



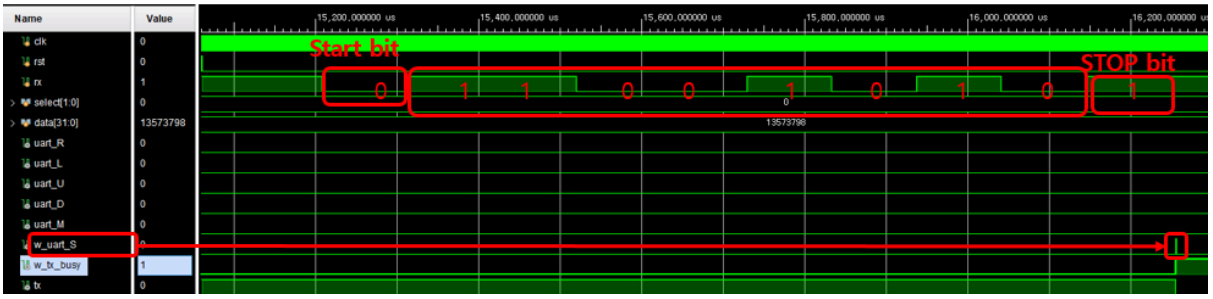
case4. uart_D 출력



case5. uart_M 출력



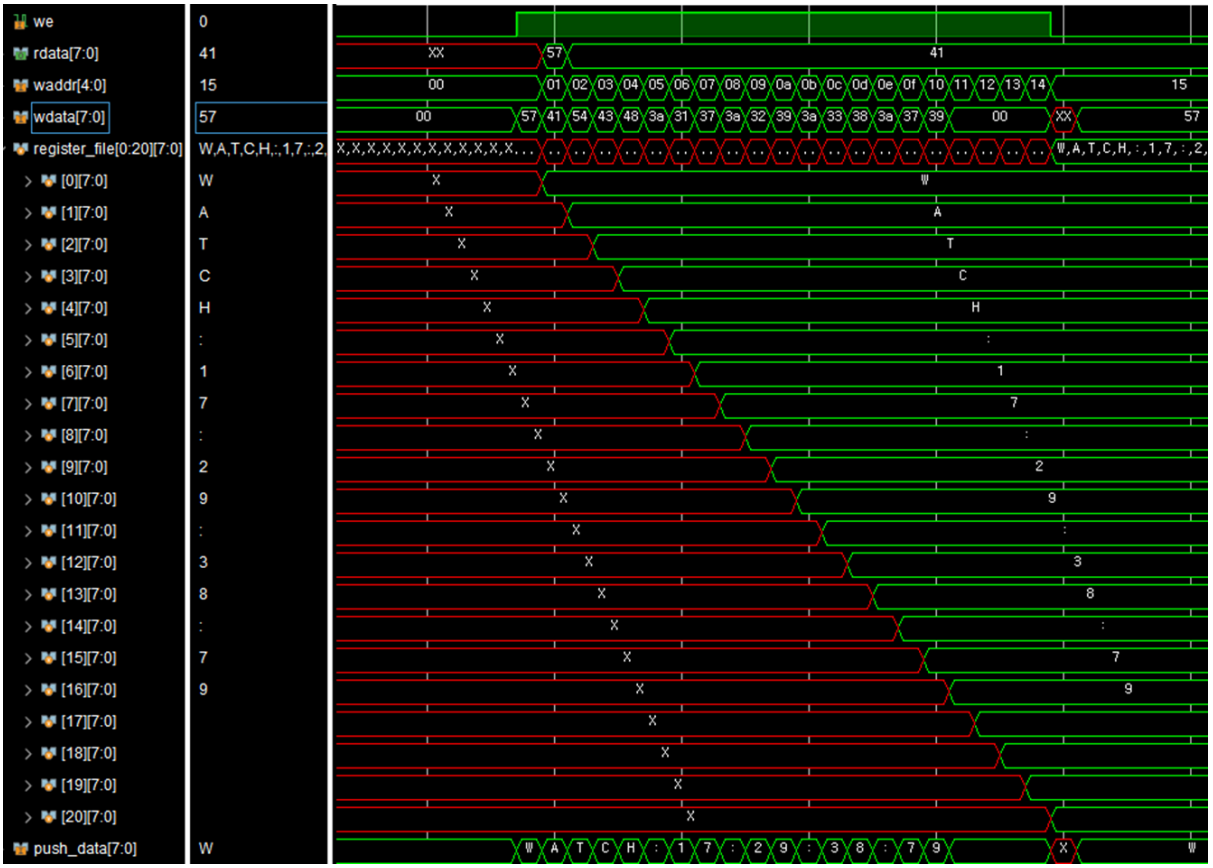
case6. uart_S 출력



5.2.5 uart_S 출력 -> data를 tx 출력

case1. watch data

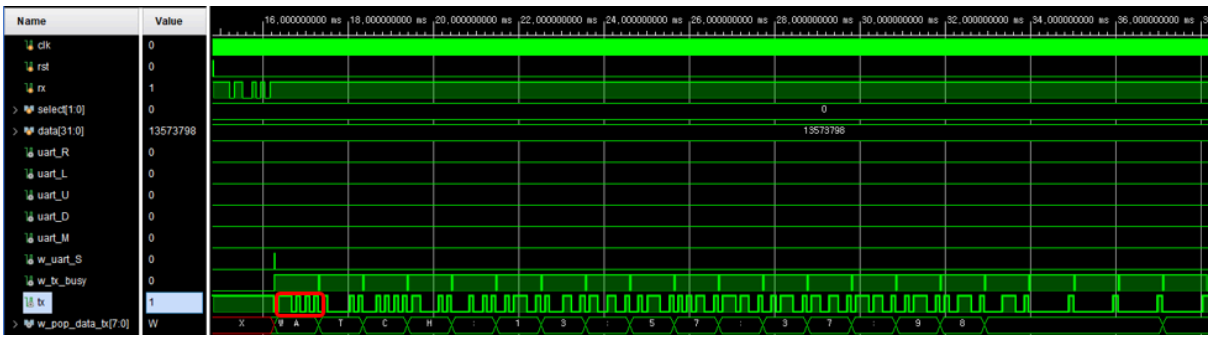
현재 watch의 시간이 13:57:37:96 이라고 가정 하였을때



uart_S를 입력 받았을 경우 watch data를 baud tick 에 맞추어 uart_tx에 전달하기 위해 다음 진리표의 값들을

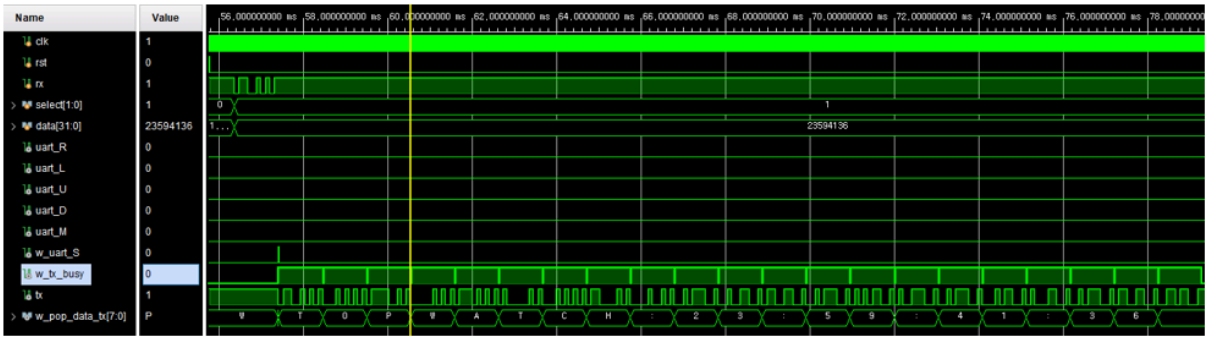
문자	Hex	Binary	LSB→MSB	문자	Hex	Binary	LSB→MSB
W	57	01010111	1 1 1 0 1 0 1 0	1	31	00110001	1 0 0 0 1 1 0 0
A	41	01000001	1 0 0 0 0 0 1 0	3	33	00110011	1 1 0 0 1 1 0 0
T	54	01010100	0 0 1 0 1 0 1 0	5	35	00110101	1 0 1 0 1 1 0 0
C	43	01000011	1 1 0 0 0 0 1 0	7	37	00110111	1 1 1 0 1 1 0 0
H	48	01001000	0 0 0 1 0 0 1 0	8	38	00111000	0 0 0 1 1 1 0 0
:	3A	00111010	0 1 0 1 1 1 0 0	9	39	00111001	1 0 0 1 1 1 0 0

fifo_tx에 저장한다.



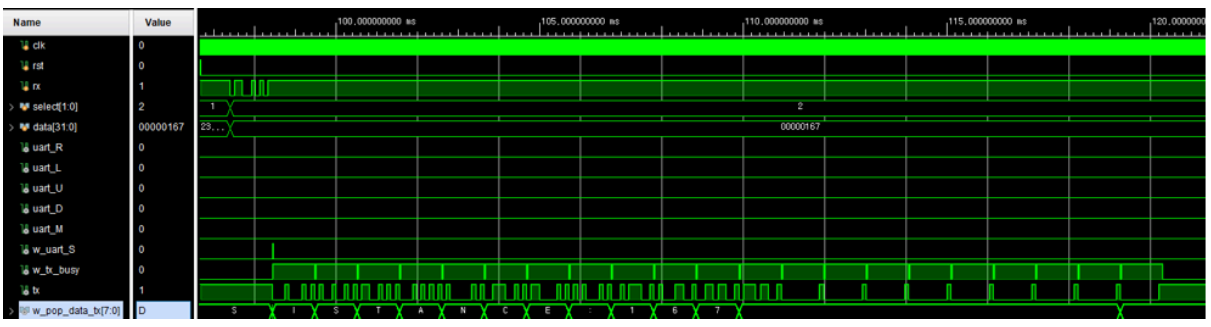
tx 출력을 baud tick의 주기에 맞춰서 출력하게 된다.

case2. stopwatch data



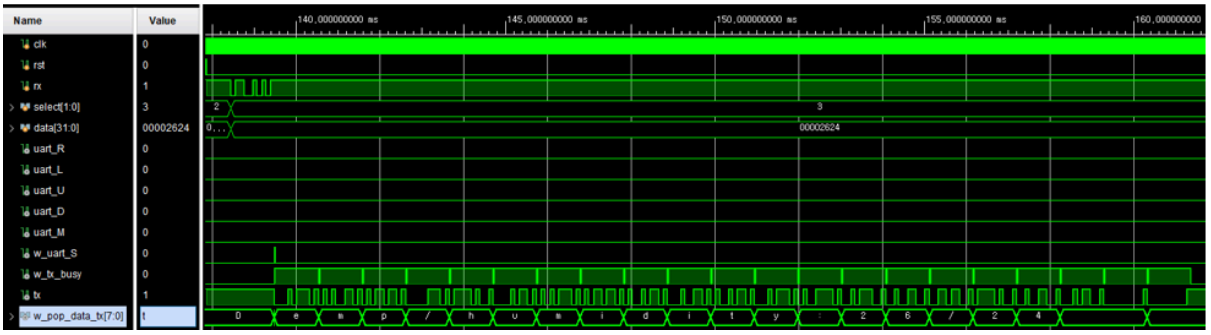
문자	Hex	Binary	LSB→MSB
S	53	01010011	11001010
T	54	01010100	00101010
O	4F	01001111	11110010
P	50	01010000	00001010
W	57	01010111	11101010
A	41	01000001	10000010
T	54	01010100	00101010
C	43	01000011	11000010
H	48	01001000	00010010
:	3A	00111010	01011100
2	32	00110010	01001100
3	33	00110011	11001100
:	3A	00111010	01011100
5	35	00110101	10101100
9	39	00110011	10011100
:	3A	00111010	01011100
4	34	00110100	00101100
1	31	00110001	10001100
:	3A	00111010	01011100
3	33	00110011	11001100
6	36	00110110	01101100

case3. SR04 data



문자	Hex	Binary	LSB→MSB	문자	Hex	Binary	LSB→MSB
D	44	01000100	0 0 1 0 0 0 1 0	C	43	01000011	1 1 0 0 0 0 1 0
I	49	01001001	1 0 0 1 0 0 1 0	E	45	01000101	1 0 1 0 0 0 1 0
S	53	01010011	1 1 0 0 1 0 1 0	:	3A	00111010	0 1 0 1 1 1 0 0
T	54	01010100	0 0 1 0 1 0 1 0	1	31	00110001	1 0 0 0 1 1 0 0
A	41	01000001	1 0 0 0 0 0 1 0	6	36	00110110	0 1 1 0 1 1 0 0
N	4E	01001110	0 1 1 1 0 0 1 0	7	37	00110111	1 1 1 0 1 1 0 0

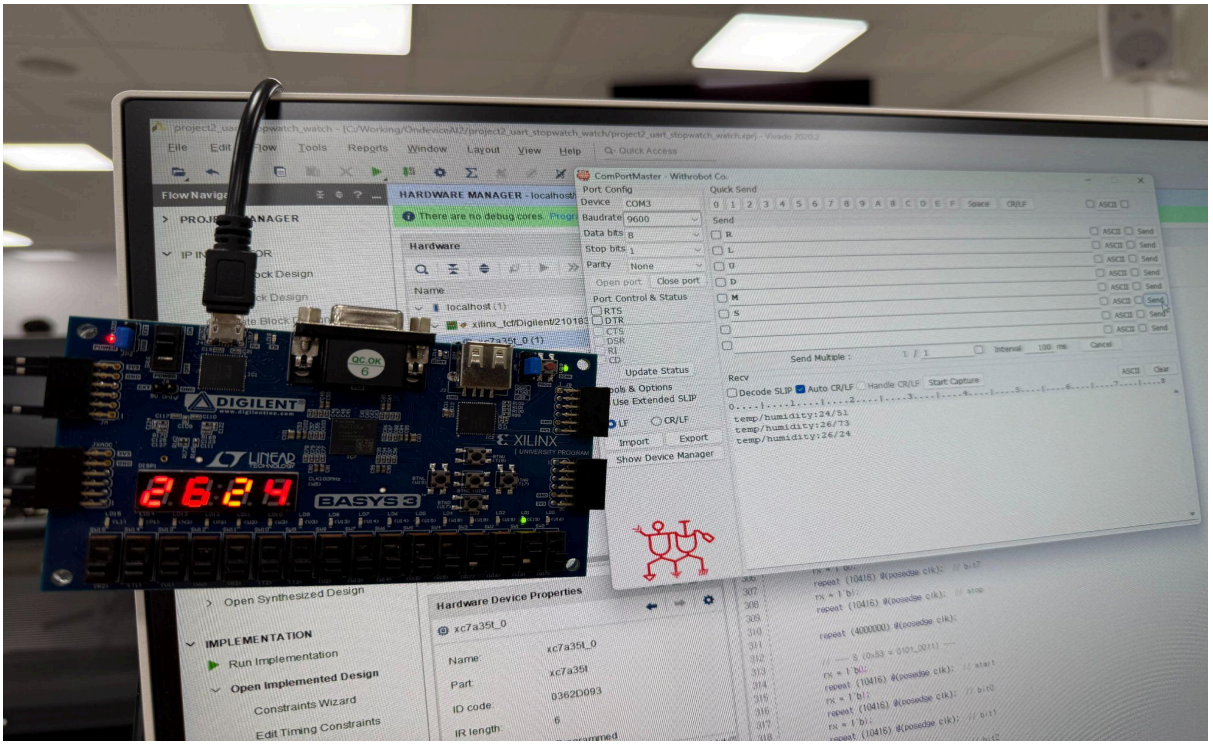
case4. DHT11 data

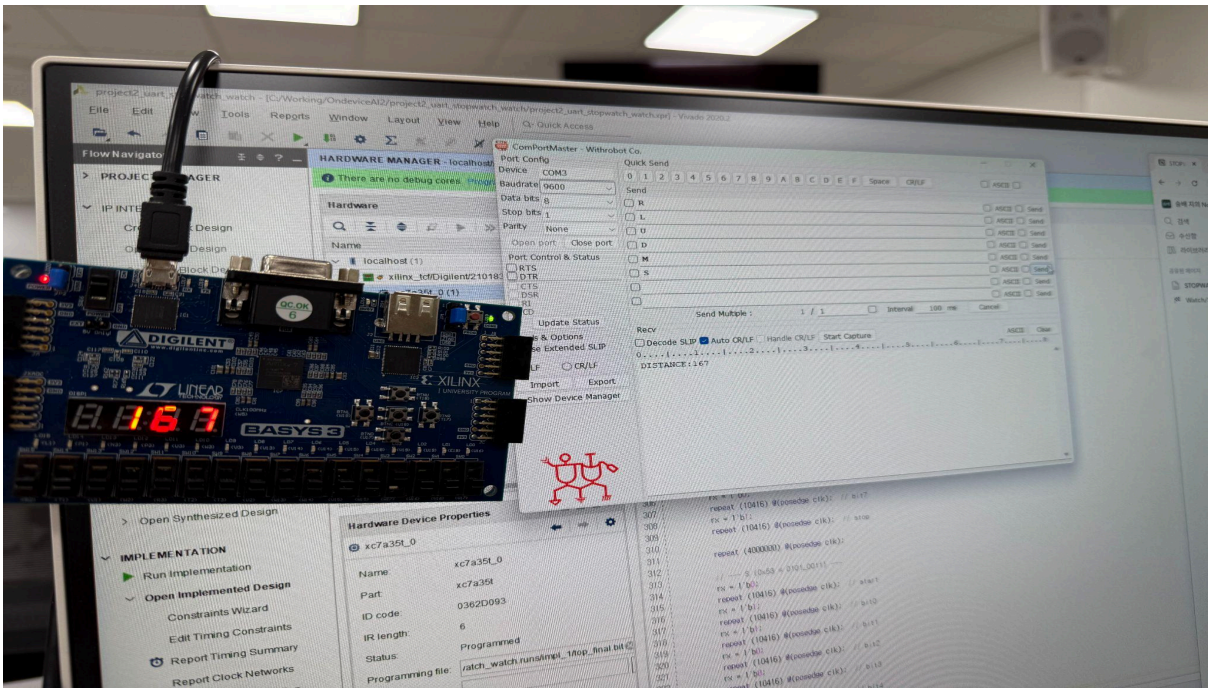


문자	Hex	Binary	LSB→MSB
T	54	01010100	00101010
E	45	01000101	10100010
M	4D	01001101	10110010
P	50	01010000	00001010
/	2F	00101111	11110100
H	48	01001000	00010010
U	55	01010101	10101010
M	4D	01001101	10110010
I	49	01001001	10010010
D	44	01000100	00100010
I	49	01001001	10010010
T	54	01010100	00101010
Y	59	01011001	10011010
:	3A	00111010	01011100
2	32	00110010	01001100
6	36	00110110	01101100
/	2F	00101111	11110100
2	32	00110010	01001100
4	34	00110100	00101100

6. 결과 분석 및 트러블슈팅 (Analysis & trouble shooting)

6.1 FPGA Implementation 결과





본 설계는 Basys3 FPGA 보드에 구현하여 실제 하드웨어 환경에서 동작을 검증하였다.

Bitstream 다운로드 후 UART 통신 프로그램인 ComPortMaster를 이용하여 시스템 상태 출력 결과를 확인하였으며, 동시에 보드의 FND 표시값과 UART 출력 문자열이 정상적으로 대응되는지를 함께 검증하였다.

DHT11 센서 기능 검증 결과, 보드의 FND에는 온도 및 습도 정보가 4자리 숫자로 표시되었고, ComPortMaster 수신 창에는 temp/humidity:24/51, temp/humidity:26/73, temp/humidity:26/24와 같은 형식의 문자열이 출력되는 것을 확인하였다. 이를 통해 DHT11 센서에서 수집한 온·습도 데이터가 내부 데이터 경로를 거쳐 FND 출력과 UART ASCII 출력으로 모두 정상 전달됨을 확인할 수 있었다.

SR04 초음파 센서 기능 검증 결과, 보드의 FND에는 측정된 거리값이 표시되었으며, ComPortMaster 수신 창에는 DISTANCE:167과 같은 문자열이 출력되는 것을 확인하였다. 이는 echo pulse width를 기반으로 계산된 거리 데이터가 정상적으로 ASCII 문자열로 변환되어 UART TX 경로로 송신되었음을 보여준다.

즉, FPGA implementation 결과를 통해 다음 사항을 확인하였다.

- DHT11 온습도 데이터가 실제 하드웨어 환경에서 정상적으로 수집됨
- SR04 거리 데이터가 실제 하드웨어 환경에서 정상적으로 측정됨
- 센서 데이터가 FND 출력과 UART 출력에 동시에 반영됨
- ASCII Sender, TX FIFO, UART Tx를 포함한 상태 출력 경로가 정상 동작함
- 통합 시스템이 보드 입력, 센서 입력, 출력 인터페이스를 포함한 전체 구조에서 안정적으로 동작함

이 결과를 통해 본 시스템은 시뮬레이션 수준을 넘어 실제 FPGA 보드 환경에서도 센서 데이터 수집, 내부 데이터 처리, FND 표시, UART 상태 출력 기능을 정상적으로 수행함을 확인하였다.

6.2 Trouble Shooting

1. SR04 초음파 센서 HW 불량

초기 FPGA 구현 과정에서 SR04 센서를 이용한 거리 측정이 정상적으로 이루어지지 않는 문제가 발생하였다. Simulation에서는 trig 신호 생성, 상태 전이, echo pulse width 기반 거리 계산 구조가 정상적으로 동작하였으나, 실제 보드 환경에서는 distance 값이 갱신되지 않거나 비정상적인 값으로 고정되는 현상이 나타났다.

문제 원인을 분석하기 위해 먼저 RTL 및 FSM 구조를 재검토하고, trig 출력과 echo 입력 조건을 중심으로 simulation 결과와 hardware 동작을 비교하였다. 그 결과 논리 설계 자체에는 큰 문제가 없었으며, 실제 센서 입력이 예상대로 들어오지 않는 점에 주목하게 되었다. 이후 배선 상태, 전원 연결, 포트 연결을 다시 점검하였으나 구조적인 오류는 발견되지 않았고, 최종적으로 사용 중이던 SR04 초음파 센서 자체가 정상 응답을 내지 못하는 HW 불량 상태였음을 확인하였다.

이에 따라 해당 문제는 설계 오류가 아니라 센서 하드웨어 결함에 의한 것으로 판단하였다. 즉, FPGA 내부 제어 로직은 정상적으로 동작하였지만, 외부 센서가 올바른 echo 신호를 반환하지 못하여 실제 거리 측정 결과가 비정상적으로 나타난 것이다. 이를 통해 센서 기반 시스템에서는 simulation 결과만으로 모든 문제를 판단할 수 없으며, 실제 하드웨어 부품의 상태와 신뢰성 또한 반드시 함께 검증해야 함을 확인하였다.

2. ASCII_DECODER 조합 논리 변경

초기 설계 하였던 ASCII_DECODER는 FSM 을 이용한 순차 논리 회로로 제작하였다. 하지만 구현 과정에서 ASCII_DECODER의 출력은 즉시 출력 용도로 상태 저장이 꼭 필요하지 않았는데, 상태 전이가 먼저 일어나거나 출력이 한 사이클 늦는 문제가 발생하였다. 이로 인해 원하는 시점에 one-hot 출력이 나오지 않았고, 디코딩 결과가 불안정하게 보였다.
이를 해결하기 위해 조합 디코더 형태로 변경하여 입력 즉시 one-hot 출력이 나오도록 수정하였다.

문제 원인

- 상태 저장형 FSM으로 구현되어 즉시성이 떨어짐.
- 상태 전이 지연으로 인해 출력 타이밍이 어긋남.
- 단순 디코더인데도 FSM 구조를 사용해 불필요하게 복잡해짐.

개선 결과

- 조합 디코더로 변경.
- 입력 즉시 one-hot 출력 구현.
- 타이밍 문제와 구조 복잡도 동시 해결.

3. FIFO_TX empty 상태 pop 판정 및 포인터 복구 실패 문제

FIFO_TX에서는 empty 상태에서 pop 판정이 충분히 엄격하지 않아 비정상적인 read 동작이 발생할 수 있었고, 전송 이후 wptr과 rptr이 정상적으로 초기화되지 않는 문제도 발생하였다. 이는 저장 데이터가 [31:0] 전체가 아니라 [20:0] 범위를 기준으로 동작하고 있었기 때문에 pop 이후 포인터 복구 조건이 예상과 다르게 작동했고, 이로 인해 두번째 status 부터 FIFO 동작 이상이 발생했다. 이를 해결하기 위해 전송 완료 후 포인터 reset 조건을 추가하고, 데이터 폭과 FIFO 접근 범위를 일치시키도록 수정하여 empty 상태에서도 안정적으로 초기화하도록 개선했다.

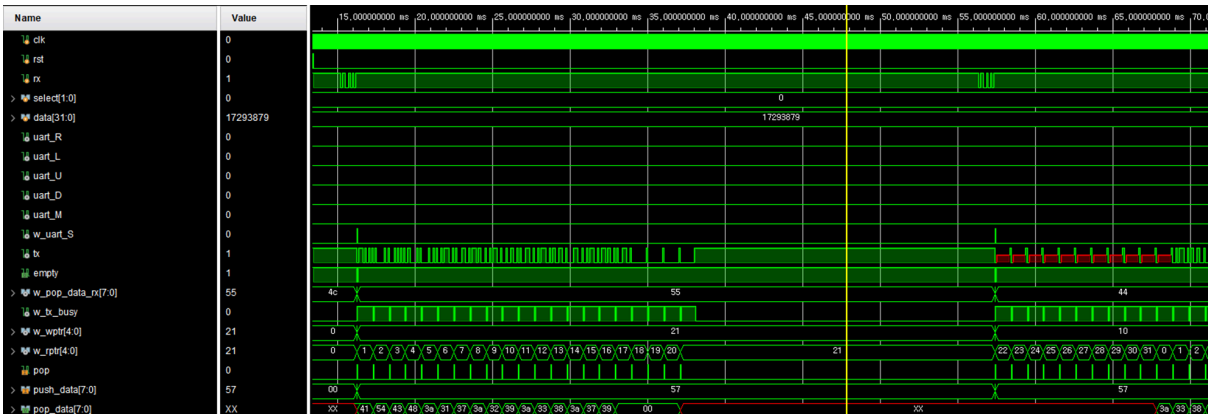
문제 원인

- empty 상태에서 pop 제어 조건이 불충분했음.
- 전송 이후 wptr, rptr 초기화 조건이 누락되었음.
- 데이터 저장 범위와 포인터 복구 조건이 서로 맞지 않았음.

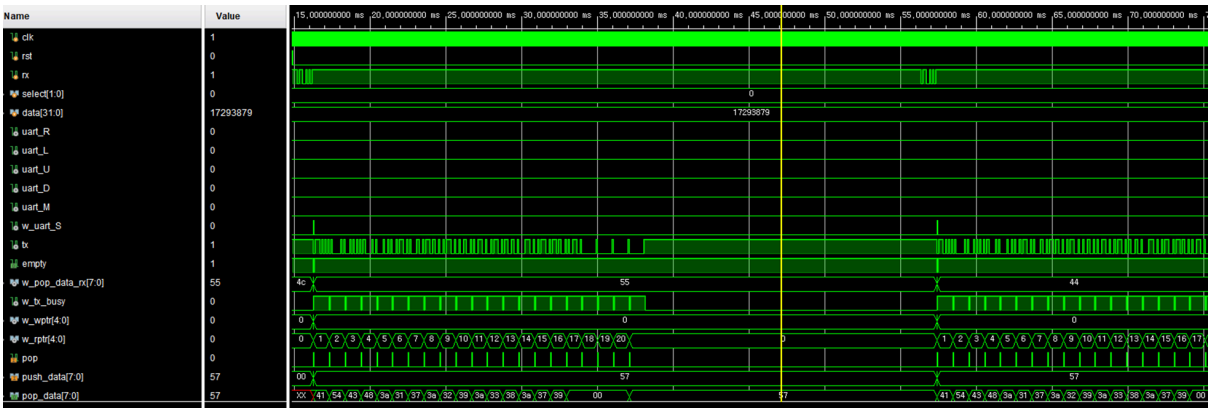
개선 내용

- 전송 완료 후 포인터 reset 조건 추가.
- 데이터 비트 범위와 FIFO 접근 구조 재정리.
- 반복 전송 시에도 FIFO가 정상 상태로 복귀하도록 수정.

<수정 전>



<수정 후>



4. ASCII_SENDER multiple driven net 문제

ASCII_SENDER에서는 문자열 생성과 송신 제어가 하나의 구조 안에 함께 들어가 있어 data_reg 갱신, bit_cnt 증가, push_data 출력이 동시에 얽히는 문제가 발생했다. 이 구조는 하나의 신호를 여러 경로에서 동시에 제어하는 형태가 되어 multiple driven net 문제를 유발했고, 데이터 흐름 또한 복잡해져 동작 구분이 불명확해졌다. 이를 해결하기 위해 문자열 생성부와 송신 제어부를 분리하고, 미리 데이터를 준비한 뒤 상태머신이 실제 송신만 담당하도록 구조를 단순화했다.

문제 원인

- 문자 생성과 송신 제어가 한 블록에 혼재되어 있었음.
- data_reg, bit_cnt, push_data가 동시에 갱신되며 충돌 가능성이 발생했음.
- 데이터 준비 시점과 송신 시작 시점의 구분이 불명확했음.
-

개선 내용

- 문자열 생성부와 송신 제어부 분리.
- 데이터 준비와 송신 타이밍을 단계적으로 구분.
- multiple driven net 문제를 해소하고 회로 구조를 단순화.

7. 결론

본 프로젝트에서는 UART, FIFO, Stopwatch/Watch, SR04, DHT11, FND 출력을 하나의 FPGA 기반 시스템으로 통합 구현하였다. 입력 단계에서는 버튼과 UART 명령을 공통 제어 신호로 처리하고, 처리 단계에서는 시간 데이터와 센서 데이터를 각각 독립적으로 생성하며, 출력 단계에서는 이를 FND와 UART를 통해 동시에 확인할 수 있도록 설계하였다. 이를 통해 개별 기능 구현을 넘어, 입력-처리-출력이 연결된 통합형 디지털 시스템 구조를 구현할 수 있었다.

또한 UART Rx/Tx와 FIFO를 이용하여 외부 터미널 기반 제어 및 상태 모니터링 기능을 추가함으로써, 단순한 보드 내 동작 확인을 넘어 사용자와 시스템 간의 양방향 데이터 흐름 구조를 구성하였다. Stopwatch/Watch 기능은 시간 생성과 제어, 기록 및 표시 기능을 포함하도록 구현하였고, SR04와 DHT11은 각각 거리 및 온습도 데이터를 수집하여 동일한 시스템 안에서 활용할 수 있도록 통합하였다.

시뮬레이션을 통해 각 기능 블록의 동작을 검증한 뒤, 실제 FPGA implementation을 통해 센서 데이터와 시간 데이터가 FND와 UART 출력으로 정상 반영되는 것을 확인하였다. 이 과정에서 FSM 기반 설계, FIFO를 이용한 버퍼 구조, 비동기 입력 처리, 센서 프로토콜 제어, multiplexing 출력 구조 등 다양한 디지털 시스템 설계 요소를 실제로 적용하고 검증할 수 있었다.

결과적으로 본 프로젝트를 통해 모듈화된 설계의 중요성, 통합 시스템에서의 데이터 흐름 관리, 그리고 simulation과 hardware 검증을 병행하는 과정의 필요성을 확인할 수 있었다. 특히 기능별 독립성과 상위 통합 구조를 함께 고려하는 설계 방식이 복합 시스템 구현에 효과적임을 확인하였으며, 향후 더 확장된 UART 기반 임베디드 시스템 설계의 기초가 되는 경험을 얻을 수 있었다.